

Universidade do Estado do Rio de Janeiro
Instituto Politécnico – IPRJ
Engenharia da Computação
Princípios de Engenharia de Software

CursoLab

Plataforma de Cursos Online

Relatório Final de Engenharia de Software

Aluno:
Caio Bento

Professor:
Dener Santos

Nova Friburgo – RJ
2026

RESUMO

O CursoLab é uma plataforma web de cursos online que cobre o ciclo completo de um curso: catálogo, compra, estudo por vídeos, apostilas e quizzes, progresso, certificado em PDF e suporte, além de um backoffice para o instrutor manter o conteúdo e atender os alunos. O sistema está publicado na nuvem, com front-end em React, API em Node.js com Express executada como função serverless na Vercel e dados, autenticação e arquivos no Supabase (PostgreSQL, Auth e Storage). Este relatório apresenta o desenvolvimento do projeto sob a perspectiva dos conceitos de Engenharia de Software estudados na disciplina: processo e ciclo de vida, metodologias, engenharia e gerenciamento de requisitos, casos de uso, arquitetura e modelo de dados, autenticação e autorização, segurança e consistência, verificação, validação e testes, manutenção, documentação, padrões de desenvolvimento e reuso. Destacam-se as regras de negócio garantidas no servidor, a rastreabilidade entre requisitos, código e testes, demonstrada em mudanças reais de requisito, e uma suíte de 158 verificações automatizadas executada localmente e contra a produção. O trabalho também registra os desafios de integração enfrentados, as limitações atuais e as possibilidades de evolução.

Palavras-chave: Engenharia de Software. Plataforma de cursos. Requisitos. Arquitetura. Autenticação. Testes automatizados. React. Express. Supabase. Vercel.

LISTA DE ILUSTRAÇÕES

Figura 1 – Ciclo de vida de desenvolvimento aplicado ao CursoLab. 13
Figura 2 – Fluxo de branches e Pull Requests: cada funcionalidade chega à produção por um merge na main. 15
Figura 3 – Fluxo de engenharia de requisitos adotado no projeto. 19
Figura 4 – Casos de uso do aluno. 26
Figura 5 – Casos de uso do administrador. 27
Figura 6 – Arquitetura: visão externa e de implantação. 29
Figura 7 – Arquitetura: fluxo interno de uma requisição. 30
Figura 8 – Modelo entidade-relacionamento do CursoLab. 31
Figura 9 – Sequência de autenticação, acesso a uma rota protegida e renovação da sessão. 33
Figura 10 – Sequência da correção de um quiz. 34
Figura 11 – Estados da matrícula e do chamado de suporte. 35
Figura 12 – Página inicial da plataforma. 38
Figura 13 – Tela de login e cadastro. 39
Figura 14 – Catálogo de cursos com busca e filtros. 40
Figura 15 – Checkout demonstrativo. 41
Figura 16 – Painel do aluno com o "continue de onde parou". 42
Figura 17 – Player de aulas com vídeo, crédito, descrição, material e grade do curso. 43
Figura 18 – Quiz no player. 44
Figura 19 – Aba de certificados do painel do aluno. 45
Figura 20 – Chamados de suporte na área do aluno. 46
Figura 21 – Fila de chamados no backoffice. 46
Figura 22 – Visão geral do backoffice. 47
Figura 23 – Editor de conteúdo do curso. 47
Figura 24 – Indicadores por curso no backoffice. 48
Figura 25 – Relação entre etapas de construção e níveis de teste no Modelo V. 53
Figura 26 – Página de detalhes do curso com a grade. 62

LISTA DE TABELAS

Tabela 1 – Metodologias estudadas e relação com o CursoLab. 16
Tabela 2 – Requisitos funcionais do ator aluno. 21
Tabela 3 – Requisitos funcionais do ator instrutor (administrador). 22
Tabela 4 – Requisitos funcionais transversais. 22
Tabela 5 – Requisitos não funcionais. 22
Tabela 6 – Mudanças de requisito registradas durante o projeto. 23
Tabela 7 – Rastreabilidade entre requisito, implementação e teste automatizado. 24
Tabela 8 – Especificação resumida do caso de uso Realizar quiz. 27
Tabela 9 – Tabelas do banco de dados. 31
Tabela 10 – Tecnologias empregadas no projeto. 36
Tabela 11 – Suítes de testes automatizados. 51
Tabela 12 – Níveis de teste e aplicação ao CursoLab. 52

SUMÁRIO

1	**INTRODUÇÃO**	**9**
1.1	Objetivo geral	9
1.2	Objetivos específicos	9
1.3	Organização do relatório	10
2	**CONTEXTO, PROBLEMA E SOLUÇÃO PROPOSTA**	**11**
2.1	Contexto do problema	11
2.2	Solução proposta	11
2.3	Escopo	12
3	**PROCESSO DE ENGENHARIA DE SOFTWARE E CICLO DE VIDA**	**13**
3.1	Planejamento	13
3.2	Requisitos	13
3.3	Design	14
3.4	Codificação e testes	14
3.5	Implantação	14
3.6	Manutenção e evolução	14
3.7	Fluxo de trabalho com Git	14
4	**METODOLOGIAS E MÉTODOS DE DESENVOLVIMENTO**	**16**
4.1	Abordagens de análise e projeto	16
4.2	Desenvolvimento iterativo	16
4.3	Metodologias estudadas e relação com o projeto	16
5	**ENGENHARIA DE REQUISITOS**	**18**
5.1	Processo de requisitos no CursoLab	19
5.1.1	Elicitação	19
5.1.2	Análise e negociação	20
5.1.3	Documentação	20
5.1.4	Verificação	20
5.1.5	Validação	20
5.2	Tipos de requisitos	20
5.2.1	Requisitos funcionais	20
5.2.2	Requisitos não funcionais	22
5.2.3	Requisitos estáveis e voláteis	23
5.2.4	Requisitos inversos	23
5.3	Gerenciamento e rastreabilidade de requisitos	24

6 CASOS DE USO 25
6.1 Caso de uso: Realizar quiz 27

7 ARQUITETURA E PROJETO DO SISTEMA 29
7.1 Visão geral 29
7.2 Camadas da aplicação 30
7.3 Modelo de dados 30
7.4 Diagramas de sequência 32
7.5 Diagramas de estados 34

8 TECNOLOGIAS UTILIZADAS 36
8.1 Justificativa das escolhas 36
8.1.1 React e Vite 36
8.1.2 Node.js e Express 36
8.1.3 Supabase 36
8.1.4 Vercel 37
8.1.5 jsPDF 37

9 DESCRIÇÃO DOS MÓDULOS E REGRAS DE NEGÓCIO 38
9.1 Área pública e autenticação 38
9.2 Catálogo e detalhes do curso 39
9.3 Carrinho, checkout e matrícula 40
9.4 Painel do aluno e "continue de onde parou" 41
9.5 Player e progressão por módulos 42
9.6 Quizzes 43
9.7 Certificado 45
9.8 Dúvidas por aula 45
9.9 Chamados de suporte 45
9.10 Backoffice e editor de conteúdo 46
9.11 Indicadores 48
9.12 Conteúdo real dos cursos 48

10 SEGURANÇA E CONSISTÊNCIA 49
10.1 Autenticação e sessão 49
10.2 Autorização 49
10.3 Separação de segredos 49
10.4 Validação de entrada 49
10.5 Transações e restrições 50

11 VERIFICAÇÃO, VALIDAÇÃO E TESTES 51

11.1 Verificação e validação 51
11.2 Análise estática e dinâmica 51
11.3 Testes automatizados 51
11.4 Níveis de teste 52
11.5 Testes funcionais e não funcionais 52
11.6 Modelo V 53

12 MANUTENÇÃO E EVOLUÇÃO 54
12.1 Manutenção corretiva 54
12.2 Manutenção evolutiva 54
12.3 Manutenção adaptativa 54
12.4 Manutenção preventiva 54
12.5 Refatoração e dívida técnica 54
12.6 Versionamento 55

13 DOCUMENTAÇÃO DE SOFTWARE 56
13.1 Documentação técnica 56
13.2 Documentação de API 56
13.3 Guia de implantação e instalação 56
13.4 Documentação do usuário 56

14 PADRÕES DE DESENVOLVIMENTO 58
14.1 Padrões criacionais 58
14.2 Padrões estruturais 58
14.3 Padrões comportamentais 58
14.4 Padrão arquitetural 58
14.5 DevSecOps 59

15 REUSO DE SOFTWARE 60

16 IMPLANTAÇÃO E AMBIENTE DE EXECUÇÃO 61
16.1 Produção 61
16.2 Execução local 61
16.3 Dados de demonstração 61

17 INTERFACE E USABILIDADE 62

18 DIFICULDADES, DECISÕES E APRENDIZADOS 63
18.1 Envio de arquivos grandes 63
18.2 Transação sem suporte no cliente 63
18.3 Reordenação com restrição de unicidade 63

18.4 Express em ambiente serverless 63
18.5 Sessão que expira durante o uso 63
18.6 Mudança de requisito com rastreabilidade 64
18.7 Aprendizados 64

19 LIMITAÇÕES E TRABALHOS FUTUROS 65

20 AVALIAÇÃO GLOBAL DO PROJETO 66
20.1 Atendimento ao problema proposto 66
20.2 Coerência entre requisitos e arquitetura 66
20.3 Pontos fortes 66
20.4 Pontos que exigem evolução 66

21 CONCLUSÃO 67

REFERÊNCIAS 68

1 INTRODUÇÃO

Este relatório descreve, de forma consolidada, o desenvolvimento do **CursoLab**, uma plataforma web de cursos online criada como projeto individual da disciplina de Princípios de Engenharia de Software. A plataforma cobre o ciclo completo de um curso: o visitante conhece a plataforma e o catálogo, o aluno compra um curso, estuda por vídeos, apostilas e quizzes, acompanha o próprio progresso e emite um certificado; o instrutor, por sua vez, monta o conteúdo, responde dúvidas e chamados e acompanha indicadores em um backoffice.

O sistema está publicado e em funcionamento em `https://plataforma-cursos-omega.vercel.app`, com todos os dados persistidos em banco de dados relacional. O código-fonte, a documentação técnica, as migrações do banco e os testes automatizados estão versionados no repositório do projeto.

Mais do que descrever telas, o objetivo deste documento é relacionar as decisões tomadas durante o desenvolvimento aos conceitos da disciplina: processo e ciclo de vida, métodos de desenvolvimento, engenharia de requisitos, modelagem, arquitetura, verificação, validação e testes, manutenção, documentação, padrões e reuso. Por isso, cada capítulo procura registrar não apenas *o que* foi feito, mas *por que* foi feito daquela forma e *como* isso se relaciona com o conteúdo estudado.

Um cuidado adotado ao longo de todo o texto é separar o que está implementado do que é apenas proposta de evolução. Quando uma funcionalidade é parcial ou não foi implementada, isso é dito explicitamente. Essa distinção é importante para que o relatório represente fielmente o sistema e não uma versão idealizada dele.

1.1 OBJETIVO GERAL

Desenvolver uma plataforma web que apoie todo o ciclo de um curso online, desde a matrícula até a certificação e o suporte, com dados persistidos em banco relacional, regras de negócio garantidas no servidor, publicação na nuvem e verificação por testes automatizados.

1.2 OBJETIVOS ESPECÍFICOS

Os objetivos específicos decompõem o objetivo geral em metas verificáveis. Cada um deles pode ser associado a requisitos, a trechos do código e a testes, o que facilita tanto a implementação quanto a avaliação do sistema.

- permitir cadastro, login, renovação de sessão e recuperação de senha;
- oferecer um catálogo com busca e filtros e uma página de detalhes de cada curso;

- realizar a matrícula por meio de carrinho e checkout, com prazo de acesso de seis meses e renovação;
- disponibilizar o conteúdo em módulos, com vídeos, apostilas em PDF e quizzes;
- liberar os módulos conforme a progressão do aluno e corrigir os quizzes no servidor;
- retomar o estudo a partir da última aula acessada;
- emitir certificado em PDF quando o aluno concluir todas as aulas com nota final de pelo menos 70%;
- oferecer dúvidas por aula e chamados de suporte;
- disponibilizar ao instrutor um backoffice para criar cursos, montar o conteúdo e acompanhar indicadores;
- publicar o sistema na nuvem, com implantação automática a cada mudança aprovada;
- garantir o comportamento das regras críticas por meio de testes automatizados.

1.3 ORGANIZAÇÃO DO RELATÓRIO

O Capítulo 2 apresenta o contexto, o problema e a solução proposta. Os Capítulos 3 e 4 tratam do processo de desenvolvimento e das metodologias. O Capítulo 5 aborda a engenharia de requisitos, e o Capítulo 6, os casos de uso. Os Capítulos 7 a 9 descrevem a arquitetura, as tecnologias, os módulos e as regras de negócio. O Capítulo 10 trata de segurança e consistência, e o Capítulo 11, de verificação, validação e testes. Os Capítulos 12 a 15 discutem manutenção, documentação, padrões e reuso. Os Capítulos 16 e 17 tratam da implantação e da interface. Por fim, os Capítulos 18 a 21 registram as dificuldades, as limitações, a avaliação global e a conclusão do trabalho.

2 CONTEXTO, PROBLEMA E SOLUÇÃO PROPOSTA

2.1 CONTEXTO DO PROBLEMA

Cursos online tornaram-se uma das formas mais comuns de estudar. Do ponto de vista de quem oferece um curso, porém, uma plataforma de ensino precisa resolver muito mais do que a exibição de vídeos. É preciso saber quem comprou o curso e até quando pode assistir, em que ordem o aluno deve avançar, como ele demonstra que aprendeu, que critério dá direito a um certificado e como o instrutor acompanha e atende os alunos.

Quando essas necessidades são resolvidas de forma improvisada, surgem problemas conhecidos. Vídeos publicados em uma página simples não registram o progresso, então o aluno não sabe onde parou e o instrutor não sabe quem está estudando. Sem controle de matrícula, o conteúdo pode ser acessado sem pagamento. Uma avaliação corrigida no próprio navegador expõe o gabarito a qualquer pessoa que abra as ferramentas de desenvolvedor. Um certificado sem regra clara perde valor. E dúvidas e pedidos de suporte espalhados por e-mail e aplicativos de mensagem geram retrabalho e respostas perdidas.

Esses problemas têm uma característica em comum: todos dependem de regras que precisam ser aplicadas de forma confiável. Não basta esconder um botão na tela; a regra precisa ser verificada no servidor, porque a interface pode ser manipulada pelo usuário. Esse ponto orientou boa parte das decisões de arquitetura do projeto.

2.2 SOLUÇÃO PROPOSTA

O CursoLab é uma aplicação web acessível pelo navegador, organizada em três frentes:

- **aprender:** catálogo com busca e filtros, compra de cursos, player com vídeo, apostila em PDF e quiz por módulo;
- **comprovar:** progresso registrado aula a aula, nota final calculada a partir dos quizzes e certificado em PDF com código de verificação;
- **gerenciar:** backoffice do instrutor para criar cursos, montar módulos, aulas e quizzes, enviar arquivos, responder dúvidas e chamados e acompanhar indicadores.

A ideia central da solução é que **toda regra importante é decidida no servidor**. Quem pode assistir a uma aula, qual é a nota de um quiz e quem tem direito a um certificado são decisões tomadas pela API, que consulta o banco de dados. A interface apenas apresenta o resultado.

Os seis cursos do catálogo possuem conteúdo real: cada curso tem três módulos, e cada módulo reúne três vídeos de canais educacionais do YouTube, incorporados com crédito ao canal, um quiz de cinco perguntas e uma apostila em PDF escrita para a plataforma.

2.3 ESCOPO

O escopo foi delimitado para que o projeto pudesse ser entregue completo e funcionando dentro do prazo da disciplina. Fazem parte do escopo a área pública, a área do aluno e o backoffice, com todos os fluxos persistidos em banco. Ficaram fora do escopo, e são tratados como trabalhos futuros no Capítulo 19, a integração com um gateway de pagamento real (o checkout é demonstrativo), a avaliação dos cursos pelos alunos, notificações por e-mail e o suporte a vários instrutores com cursos próprios.

3 PROCESSO DE ENGENHARIA DE SOFTWARE E CICLO DE VIDA

Um processo de engenharia de software é um conjunto organizado de atividades que orienta o desenvolvimento e a evolução de um sistema, envolvendo especificação, projeto, implementação, validação e evolução. No CursoLab, o desenvolvimento pode ser representado pelas seis etapas gerais do ciclo de vida: planejamento, requisitos, design, codificação e testes, implantação e manutenção.

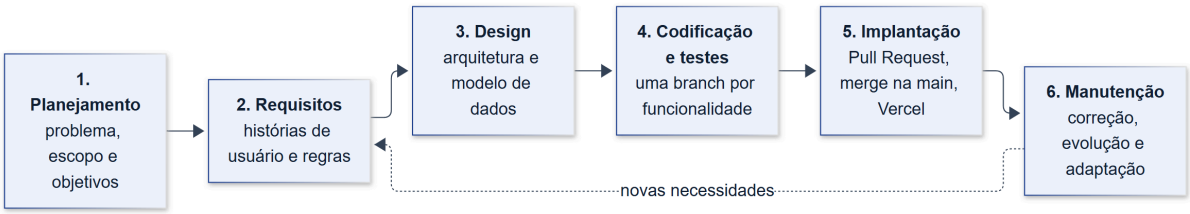


Figura 1 – Ciclo de vida de desenvolvimento aplicado ao CursoLab.

A representação por etapas não significa que o projeto tenha seguido um fluxo linear. Cada funcionalidade percorreu o ciclo inteiro e, depois de publicada, gerou novas necessidades que voltaram para a etapa de requisitos, como indica a seta tracejada da figura. A regra do certificado é um exemplo: ela foi implementada, publicada, revista e alterada, com impacto controlado sobre o banco, a API, as telas, os testes e a documentação.

3.1 PLANEJAMENTO

No planejamento, a ideia ampla de uma "plataforma de cursos" foi transformada em um escopo realizável. O critério foi priorizar o fluxo central de valor: conhecer um curso, comprá-lo, estudar, ser avaliado e receber o certificado, além das ferramentas mínimas para o instrutor manter o conteúdo. Funcionalidades como pagamento real e avaliações de cursos foram adiadas, pois aumentariam o escopo sem alterar a demonstração do fluxo principal.

O planejamento também definiu a estratégia de infraestrutura: usar serviços gerenciados (Supabase para banco, autenticação e arquivos, e Vercel para hospedagem) para reduzir o esforço com servidores e concentrar o trabalho nas regras do domínio.

3.2 REQUISITOS

Os requisitos foram levantados como histórias de usuário dos dois atores principais, aluno e instrutor, e depois detalhados em requisitos funcionais e não funcionais com identificação, situação e rastreabilidade. O processo é detalhado no Capítulo 5.

3.3 DESIGN

No design, as necessidades foram traduzidas em estrutura de software: a divisão entre front-end, API e banco de dados; o modelo de dados com suas tabelas, relacionamentos e restrições; e a definição de onde cada regra seria verificada. A opção foi por uma aplicação monolítica modularizada, com uma única API e um único banco, publicada como função serverless. O Capítulo 7 descreve essa arquitetura.

3.4 CODIFICAÇÃO E TESTES

A codificação foi incremental: cada funcionalidade foi construída, testada e publicada antes da próxima. Para as regras críticas, foram escritos testes automatizados de integração, que chamam a API real e conferem o comportamento esperado, inclusive os casos que não podem acontecer, como um visitante recebendo o gabarito de um quiz. Antes de cada publicação eram executados a análise estática (ESLint), o build e a suíte de testes.

3.5 IMPLANTAÇÃO

A implantação foi tratada desde o início do projeto. O repositório está conectado à Vercel, de modo que cada merge na branch principal publica automaticamente uma nova versão do site e da API. As mudanças no banco de dados são feitas por migrações SQL numeradas e versionadas junto com o código. Depois de cada publicação, a suíte de testes era executada novamente contra o endereço de produção.

3.6 MANUTENÇÃO E EVOLUÇÃO

Como o sistema esteve publicado durante o desenvolvimento, a manutenção ocorreu em paralelo à construção: correções, novas funcionalidades e adaptações ao ambiente de execução. O Capítulo 12 apresenta exemplos reais de cada tipo de manutenção.

3.7 FLUXO DE TRABALHO COM GIT

O controle de versão foi a principal evidência do processo. Nenhuma mudança foi feita diretamente na branch principal. Cada funcionalidade foi desenvolvida em uma branch própria (por exemplo, `feat/prazo-de-acesso`), enviada como Pull Request com descrição e checklist de verificação e, depois de aprovada, incorporada à branch `main`, o que disparava a publicação automática.

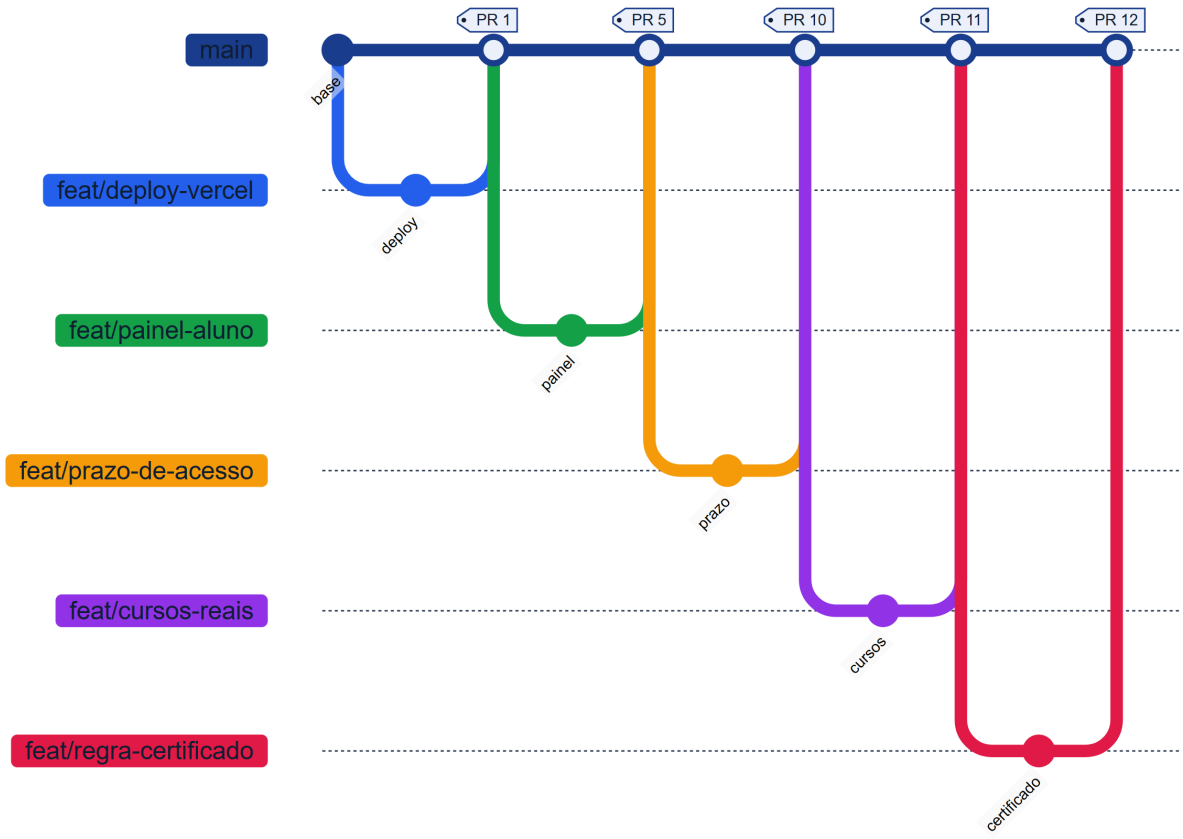


Figura 2 – Fluxo de branches e Pull Requests: cada funcionalidade chega à produção por um merge na main.

Ao final do projeto, o histórico registrava 12 Pull Requests, 37 commits na branch principal e 15 migrações de banco de dados. Esse histórico permite responder, para qualquer mudança, quando ela ocorreu, o que foi alterado e qual era a justificativa.

4 METODOLOGIAS E MÉTODOS DE DESENVOLVIMENTO

4.1 ABORDAGENS DE ANÁLISE E PROJETO

Entre as abordagens estudadas estão a Análise e Projeto Orientados a Objetos (APOO) e a Análise e Projeto Estruturados (AEPE). O CursoLab se aproxima mais da modelagem orientada ao domínio: o sistema foi pensado a partir de entidades como curso, módulo, aula, quiz, matrícula, progresso, dúvida e chamado, com responsabilidades e relacionamentos bem definidos. Ao mesmo tempo, como a implementação usa JavaScript com funções e um banco relacional, a modelagem foi documentada principalmente por diagramas de casos de uso, de sequência, de estados e entidade-relacionamento.

4.2 DESENVOLVIMENTO ITERATIVO

O processo pode ser caracterizado como **iterativo e incremental, com práticas ágeis**. Cada iteração entregava uma funcionalidade completa (requisito, código, testes e publicação) e permitia rever requisitos a partir do uso do sistema publicado.

Não se afirma, porém, a adoção formal de Scrum. O projeto foi desenvolvido por uma única pessoa, sem papéis de Product Owner e Scrum Master, sem sprints de duração fixa e sem cerimônias formais. Dizer que o projeto usou Scrum seria confundir o uso de iterações com a adoção de uma metodologia completa.

4.3 METODOLOGIAS ESTUDADAS E RELAÇÃO COM O PROJETO

A Tabela 1 relaciona as principais metodologias estudadas na disciplina ao CursoLab.

Tabela 1 – Metodologias estudadas e relação com o CursoLab.

Metodologia	**Ideia central**	**Relação com o CursoLab**
Ágil	Iterações curtas, colaboração e adaptação a mudanças.	Relação direta: entregas incrementais e requisitos revistos a partir do uso.
Lean	Reduzir desperdício e focar em valor.	Escopo limitado ao fluxo central; serviços gerenciados evitaram trabalho com infraestrutura.
Scrum	Sprints, papéis definidos e inspeção frequente.	Não adotado formalmente; apenas as iterações são compatíveis.
Kanban	Gestão visual do fluxo de trabalho.	Os Pull Requests funcionaram como fluxo de trabalho, sem quadro formal.

DevOps	Integração entre desenvolvimento e operação.	Relação forte: publicação automática a cada merge e testes contra a produção.
FDD	Desenvolvimento orientado a funcionalidades.	Cada branch correspondeu a uma funcionalidade completa.
XP	Ciclos curtos, simplicidade e testes.	Testes automatizados e simplicidade de design; sem programação em par.
Cascata	Fases sequenciais fortemente documentadas.	Não representa o processo, pois houve revisões durante o desenvolvimento.
Prototipação	Protótipos para validar o entendimento.	Telas novas foram prototipadas e aprovadas antes da implementação final.
Espiral	Iterações guiadas por análise de riscos.	Não adotado formalmente; riscos de integração foram tratados primeiro.

5 ENGENHARIA DE REQUISITOS

A engenharia de requisitos compreende a descoberta, a análise, a documentação, a verificação e a validação das funções e restrições do sistema. Erros nessa etapa tendem a ser os mais caros, porque se propagam para o projeto, o código e os testes.

No CursoLab, grande parte da complexidade está nas regras do domínio e não nas telas. Requisitos como "assistir à aula", "fazer o quiz" ou "emitir o certificado" são insuficientes se descritos superficialmente: é preciso definir quem pode assistir, como a nota é calculada, o que acontece quando o prazo de acesso vence e qual critério libera o certificado.

5.1 PROCESSO DE REQUISITOS NO CURSOLAB

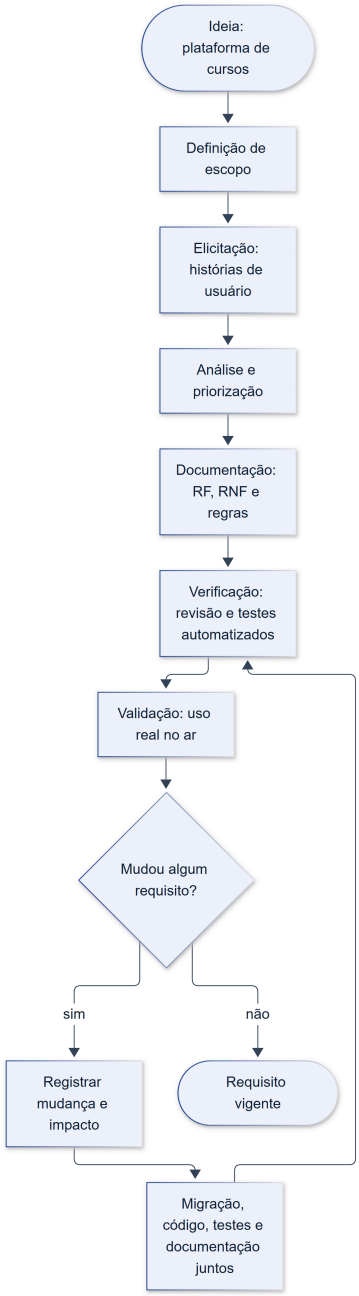


Figura 3 – Fluxo de engenharia de requisitos adotado no projeto.

5.1.1 Elicitação

Como projeto acadêmico, não houve um cliente externo formal. A elicitação foi feita por análise do domínio de ensino online, estudo de plataformas de cursos conhecidas e escrita de **histórias de usuário** no formato "Como [ator], eu quero [ação], para que [benefício]", cada uma com critérios de aceitação. Foram escritas 18 histórias: 11 do aluno e 7 do instrutor. Um exemplo:

> **HU – Emitir certificado de conclusão.** Como aluno, eu quero emitir um certificado de conclusão, para que eu possa comprovar meu aprendizado. Critérios de aceitação: o botão só é habilitado quando a regra de conclusão é atendida; o certificado é gerado em PDF; contém nome, curso, carga horária e data de conclusão.

5.1.2 Análise e negociação

Os requisitos foram priorizados pelo valor para o fluxo central e pela viabilidade dentro do prazo. A análise também identificou dependências: não há matrícula sem curso, não há progresso sem matrícula, e o certificado depende do progresso e das notas. Essas dependências definiram a ordem de implementação. Requisitos de menor prioridade, como cupons de desconto e avaliação de cursos, foram registrados como não implementados.

5.1.3 Documentação

Os requisitos foram documentados no repositório em `docs/documentacao-requisitos.md`, com histórias de usuário, requisitos funcionais e não funcionais numerados e a situação de cada um. Tabelas favorecem a rastreabilidade; o texto explica o contexto e as decisões; os diagramas dão a visão estrutural.

5.1.4 Verificação

A verificação conferiu se os requisitos eram claros, consistentes e testáveis. Um requisito como "o quiz deve ser seguro" é vago; já "o gabarito dos quizzes nunca é enviado ao navegador do aluno" pode ser verificado por um teste automatizado, e de fato é. Sempre que possível, os requisitos foram escritos de forma a originar um teste.

5.1.5 Validação

A validação procurou confirmar se o sistema construído resolve o problema. Ela foi feita com o sistema publicado: navegando como visitante, aluno e administrador, com cursos de conteúdo real, e ajustando os requisitos quando o uso mostrava uma regra inadequada. A mudança da regra do certificado, descrita na Seção 5.2.3, surgiu justamente dessa validação.

5.2 TIPOS DE REQUISITOS

5.2.1 Requisitos funcionais

Os requisitos funcionais descrevem o que o sistema deve fazer. A Tabela 2 apresenta os requisitos do aluno, a Tabela 3 os do instrutor e a Tabela 4 os transversais.

Tabela 2 – Requisitos funcionais do ator aluno.

ID	Requisito	Situação	Observação
RF01	Cadastrar-se com nome, e-mail e senha	Implementado	
RF02	Autenticar-se e manter a sessão ativa	Implementado	Sessão renovada automaticamente
RF03	Solicitar e concluir a redefinição de senha	Implementado	Link enviado por e-mail
RF04	Consultar o catálogo de cursos	Implementado	
RF05	Buscar cursos pelo título	Implementado	Resultados a cada tecla digitada
RF06	Filtrar por nível, faixa de preço e avaliação	Implementado	
RF07	Visualizar os detalhes e a grade de um curso	Implementado	Sem vídeo de apresentação
RF08	Adicionar e remover cursos do carrinho	Implementado	
RF09	Aplicar cupom de desconto	Não implementado	
RF10	Matricular-se, sem duplicar matrícula ativa	Implementado	Checkout demonstrativo (cartão ou Pix)
RF11	Consultar os cursos adquiridos e o prazo de acesso	Implementado	
RF12	Assistir às aulas e acessar o material de apoio	Implementado	Arquivo, YouTube ou Vimeo
RF13	Marcar e desmarcar a conclusão de uma aula	Implementado	
RF14	Ter os módulos liberados conforme a progressão	Implementado	
RF15	Responder quizzes e receber a nota	Implementado	Correção no servidor
RF16	Emitir certificado de conclusão em PDF	Implementado	100% das aulas e nota final ≥ 70%
RF17	Enviar dúvidas em uma aula	Parcial	Sem notificação da resposta
RF18	Avaliar o curso com nota e comentário	Não implementado	
RF19	Abrir e acompanhar chamados de suporte	Implementado	Aberto, respondido e encerrado

Tabela 3 – Requisitos funcionais do ator instrutor (administrador).

ID	Requisito	Situação	Observação
RF20	Criar um curso com título, preço, nível e prazo de acesso	Implementado	
RF21	Criar e organizar módulos	Implementado	Renomear, reordenar e excluir, em transação
RF22	Criar aulas com tipo, duração, descrição e materiais	Implementado	Envio de vídeos e PDFs pelo computador
RF23	Cadastrar quizzes com perguntas e nota mínima	Implementado	
RF24	Definir e alterar o preço de um curso	Parcial	Sem preço promocional
RF25	Responder às dúvidas dos alunos	Implementado	
RF26	Editar e excluir cursos do catálogo	Implementado	
RF27	Consultar indicadores do catálogo	Parcial	Sem série histórica de inscrições

Tabela 4 – Requisitos funcionais transversais.

ID	Requisito	Situação
RF28	Restringir as operações de gestão ao papel de administrador	Implementado
RF29	Restringir conteúdo, progresso e dúvidas a quem tem matrícula dentro do prazo	Implementado
RF30	Retomar o curso a partir da última aula acessada	Implementado
RF31	Permitir ao aluno atualizar o nome e trocar a senha	Implementado
RF32	Renovar o acesso a um curso expirado, mantendo o progresso	Implementado

5.2.2 Requisitos não funcionais

Os requisitos não funcionais definem qualidades e restrições que atravessam todo o sistema.

Tabela 5 – Requisitos não funcionais.

ID	Categoria	Requisito
RNF01	Segurança	A chave secreta do banco permanece exclusivamente no servidor.
RNF02	Segurança	O controle de acesso é verificado no servidor, não na interface.
RNF03	Segurança	A API aceita requisições apenas da origem configurada.
RNF04	Segurança	Senhas têm no mínimo 6 caracteres e são armazenadas apenas como hash.

RNF05	Segurança	O token de acesso é enviado no cabeçalho Authorization e mantido só na sessão do navegador.
RNF06	Segurança	O gabarito dos quizzes nunca é enviado ao navegador do aluno.
RNF07	Consistência	Operações que alteram várias tabelas ao mesmo tempo são atômicas.
RNF08	Desempenho	Arquivos são enviados direto ao armazenamento, sem passar pela API.
RNF09	Usabilidade	A interface é responsiva e integralmente em português do Brasil.
RNF10	Manutenibilidade	As camadas de apresentação, aplicação e dados comunicam-se apenas por HTTP.
RNF11	Disponibilidade	O sistema é hospedado na nuvem e publicado automaticamente a cada mudança aprovada.

5.2.3 Requisitos estáveis e voláteis

Requisitos estáveis tendem a permanecer válidos ao longo do projeto, como a existência de cursos, módulos, aulas e matrículas. Requisitos voláteis têm maior probabilidade de mudar, geralmente porque dependem de decisões de negócio. O CursoLab teve três mudanças relevantes de requisito durante o desenvolvimento, registradas na Tabela 6.

Tabela 6 – Mudanças de requisito registradas durante o projeto.

Requisito	**Antes**	**Depois**	**Artefatos alterados juntos**
Liberação do certificado	60% das aulas concluídas	100% das aulas e nota final ≥ 70% (média da melhor nota de cada quiz)	migração 015, API, painel, player, landing, PDF, testes e documentação
Acesso ao curso	sem prazo	6 meses por matrícula, com renovação	migração 013, API, painel, checkout e testes
Conteúdo dos cursos	aulas de demonstração	vídeos reais com crédito, quizzes e apostilas	migração 014, editor, player e script de carga

A mudança do certificado ilustra bem por que o gerenciamento de requisitos precisa de rastreabilidade. Se apenas o texto da página inicial tivesse sido atualizado, a API continuaria liberando o certificado com 60%. Se apenas a API mudasse, a interface prometeria uma regra que não vale mais. A alteração foi feita em um único Pull Request que atualizou o banco, a API, as telas, os testes e a documentação.

5.2.4 Requisitos inversos

Requisitos inversos definem o que **não pode** acontecer. No CursoLab, eles foram formalizados e, em sua maioria, transformados em testes automatizados:

- RI01: o sistema não deve enviar o gabarito de um quiz ao navegador do aluno;

- RI02: o sistema não deve marcar um quiz como concluído sem que o aluno o tenha feito;
- RI03: o sistema não deve entregar vídeos e materiais a quem não tem matrícula dentro do prazo;
- RI04: o sistema não deve liberar o certificado com aulas pendentes ou nota final abaixo de 70%;
- RI05: o sistema não deve deixar o conteúdo de um curso salvo pela metade quando uma gravação falhar;
- RI06: o sistema não deve permitir operações de gestão a usuários que não sejam administradores;
- RI07: o sistema não deve expor a chave secreta do banco ao navegador.

5.3 GERENCIAMENTO E RASTREABILIDADE DE REQUISITOS

A rastreabilidade associa cada requisito à sua implementação e ao teste que confirma seu funcionamento. Assim, quando um requisito muda, é possível saber quais artefatos precisam ser revisados. A Tabela 7 apresenta a rastreabilidade das regras mais críticas.

Tabela 7 – Rastreabilidade entre requisito, implementação e teste automatizado.

Requisito	**Implementação**	**Teste automatizado**
RF15 / RI01	rota `quiz-attempt`; conteúdo sem gabarito para o aluno	seguranca-do-quiz: "aluno matriculado não recebe o gabarito"
RI02	rota de progresso recusa quiz	seguranca-do-quiz: "não marca quiz como concluído sem fazer (400)"
RF16 / RI04	`toProgressResponse` e rota do certificado	painel-aluno: "certificado bloqueado com aulas pendentes (403)"; seguranca-do-quiz: "certificado bloqueado pela nota (403)"
RF29 / RI03	`hasCourseAccess` e prazo de acesso	prazo-de-acesso: "acesso expirado: conteúdo volta a ser só a grade"
RF21 / RI05	função `save_course_content`	conteudo: "conteúdo intacto após payloads inválidos"
RF28 / RI06	middleware `requireAdmin`	duvidas: "aluno não acessa a lista do backoffice (403)"; chamados: "aluno não acessa lista do admin (403)"
RF30	`findNextLesson` e `last-lesson`	continuar-de-onde-parou (16 verificações)
RNF08	rota de upload com URL assinada	uploads: "PUT direto no Storage"

6 CASOS DE USO

A modelagem por casos de uso representa as funcionalidades do ponto de vista dos atores. Diferentemente do modelo de dados, que descreve a estrutura interna, o caso de uso descreve objetivos observáveis: "emitir certificado" é um objetivo do aluno; "consultar a tabela lesson_progress" é um detalhe de implementação.

O CursoLab tem dois atores principais. O **aluno** compra cursos, estuda e é avaliado. O **administrador** (instrutor) mantém o catálogo e atende os alunos. Há também o **visitante**, que acessa a página inicial, o catálogo em destaque e a grade dos cursos, mas não o conteúdo das aulas.

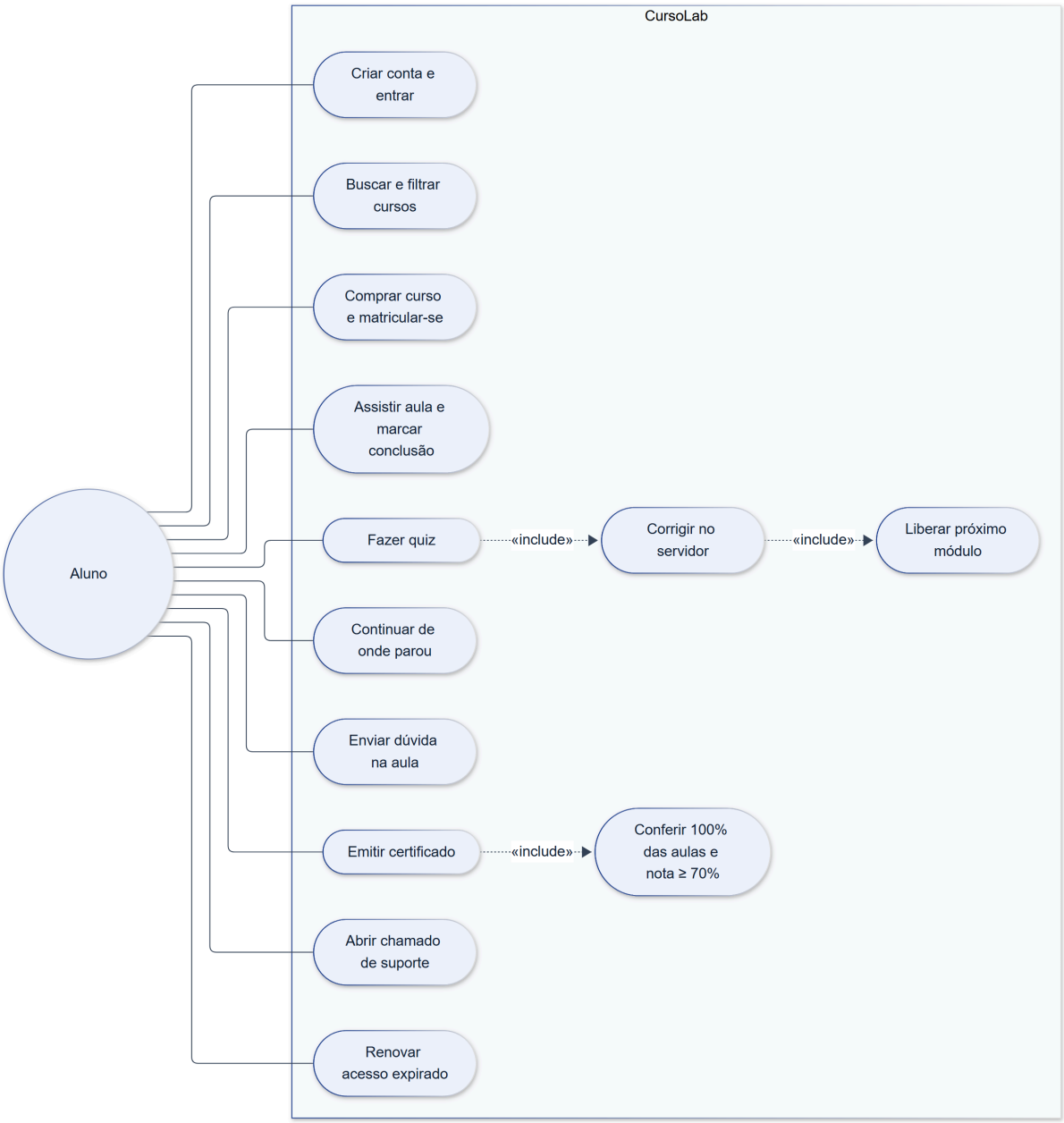


Figura 4 – Casos de uso do aluno.

As relações de inclusão (`«include»`) destacam comportamentos obrigatórios: fazer um quiz sempre inclui a correção no servidor, e a aprovação é o que libera o módulo seguinte; emitir o certificado sempre inclui a conferência da regra de conclusão.

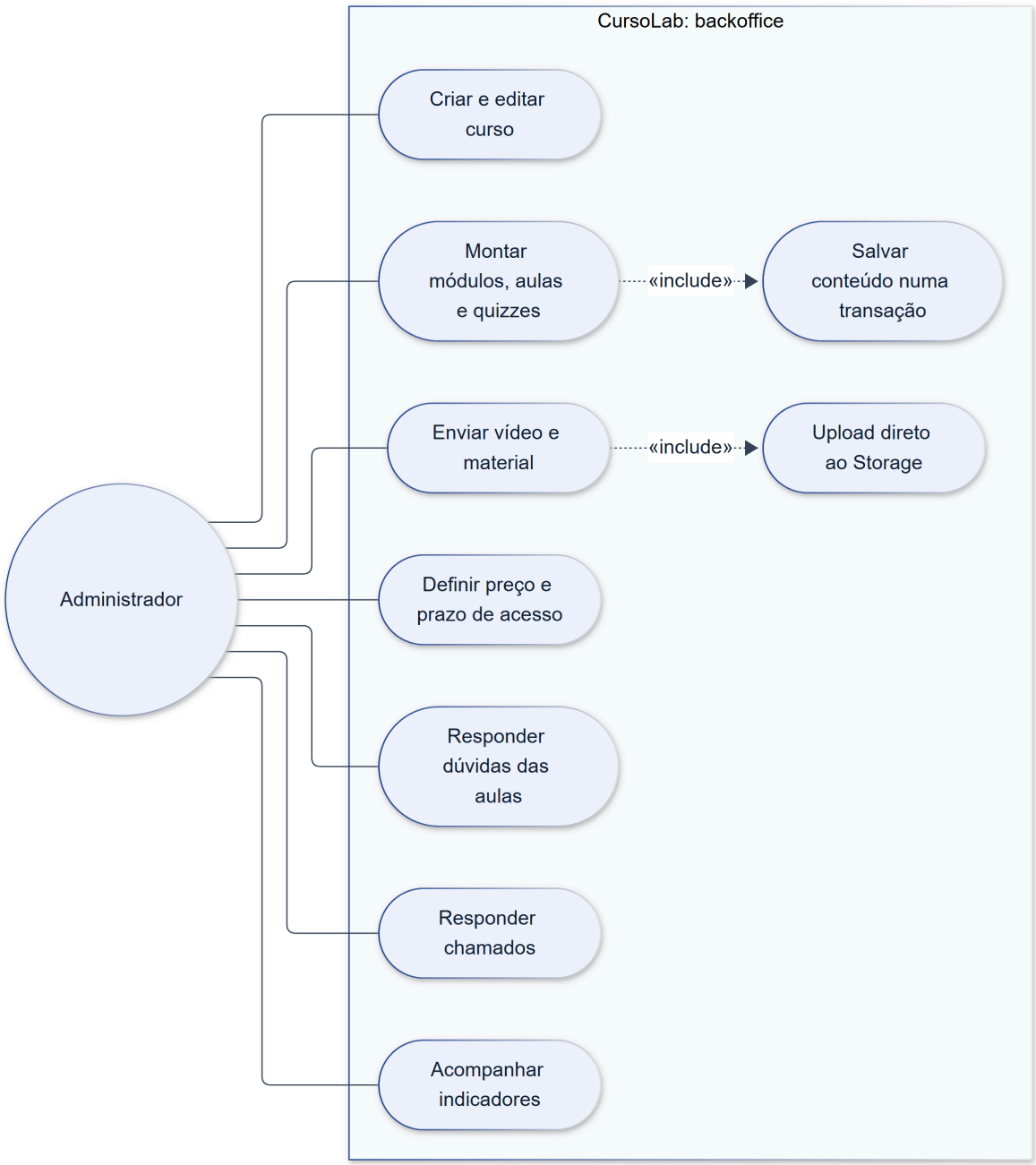


Figura 5 – Casos de uso do administrador.

6.1 CASO DE USO: REALIZAR QUIZ

O caso de uso "Realizar quiz" é representativo porque reúne autenticação, controle de acesso, regra de negócio e persistência. A Tabela 8 apresenta sua especificação resumida. Cada fluxo alternativo originou um teste automatizado.

Tabela 8 – Especificação resumida do caso de uso Realizar quiz.

Campo	**Descrição**

Ator principal	Aluno.
Pré-condições	Aluno autenticado, com matrícula ativa e dentro do prazo; módulo do quiz liberado.
Fluxo principal	O aluno responde às perguntas e envia; o sistema confere o acesso, busca o gabarito no banco, calcula a nota, grava a conclusão e a melhor nota e libera o módulo seguinte.
Pós-condições	Quiz concluído, melhor nota registrada e progresso atualizado.
Fluxos alternativos	Nota abaixo da mínima (pode refazer); matrícula expirada (403); sem matrícula (403); respostas ausentes (400); sessão inválida (401).

7 ARQUITETURA E PROJETO DO SISTEMA

7.1 VISÃO GERAL

O CursoLab é uma aplicação web **monolítica modularizada**: há um único front-end, uma única API e um único banco de dados, organizados internamente em responsabilidades separadas. A figura a seguir mostra a visão de implantação.

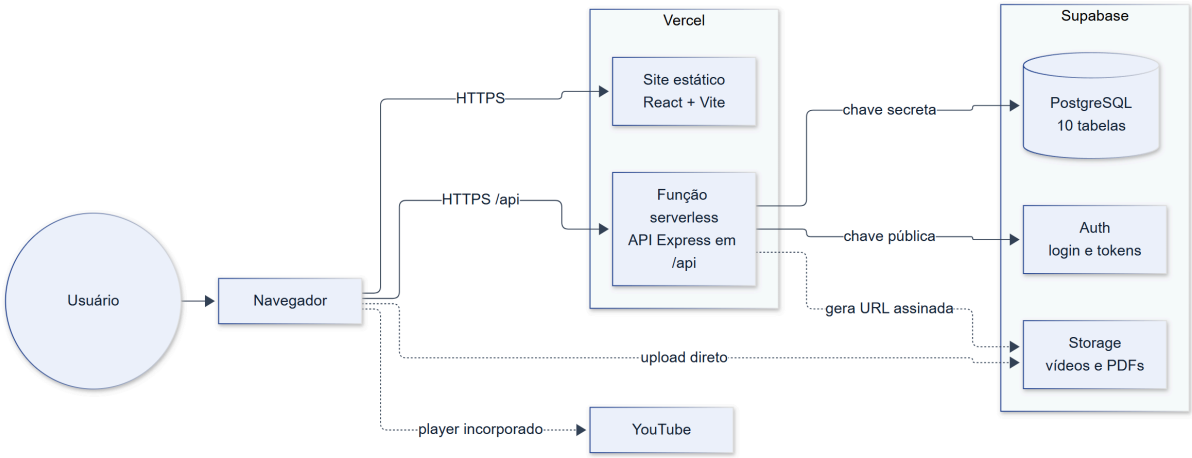


Figura 6 – Arquitetura: visão externa e de implantação.

O navegador carrega a interface, um site estático construído com React e Vite e servido pela CDN da Vercel. Todas as chamadas para endereços que começam com `/api` são direcionadas a uma função serverless, que executa a API escrita em Express. A API se comunica com o Supabase, que fornece três serviços: **Auth** (contas e tokens), **PostgreSQL** (dados) e **Storage** (vídeos e documentos). Arquivos grandes não passam pela API: ela apenas gera uma URL assinada, e o navegador envia o arquivo direto ao Storage. Os vídeos de terceiros são incorporados com o player oficial do YouTube.

A decisão por um monólito foi coerente com o porte do projeto. Separar o sistema em microsserviços traria custos de comunicação entre serviços, autenticação distribuída, implantação e observabilidade que não se justificam para uma aplicação desenvolvida por uma pessoa. Ao mesmo tempo, publicar a API como função serverless eliminou a necessidade de administrar um servidor.

7.2 CAMADAS DA APLICAÇÃO

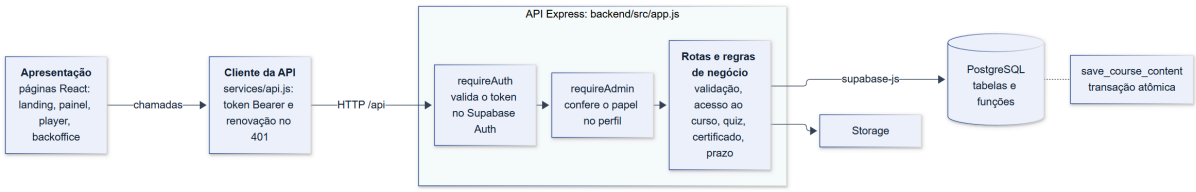


Figura 7 – Arquitetura: fluxo interno de uma requisição.

Internamente, uma requisição atravessa camadas com responsabilidades distintas:

- **Apresentação:** páginas e componentes React. Exibem dados e capturam ações, sem conhecer o banco de dados.
- **Cliente da API:** o módulo `services/api.js` concentra as chamadas HTTP, anexa o token de acesso e renova a sessão automaticamente quando ela expira.
- **Controle de acesso:** os middlewares `requireAuth` (valida o token no Supabase Auth) e `requireAdmin` (confere o papel no perfil) são aplicados às rotas antes de qualquer regra.
- **Rotas e regras de negócio:** validam a entrada, verificam o acesso ao recurso (por exemplo, matrícula dentro do prazo) e aplicam as regras do domínio.
- **Dados:** o PostgreSQL guarda os dados e aplica restrições de integridade; operações compostas são executadas em funções do banco, dentro de transações.

Essa separação reduz o acoplamento. Uma mudança na regra do certificado alterou a API e os textos das telas, mas não exigiu mudar o modo como as telas obtêm os dados. Do mesmo modo, os testes de integração verificam a API sem depender da interface.

7.3 MODELO DE DADOS

O banco possui 10 tabelas, que podem ser agrupadas em quatro áreas: pessoas, catálogo, aprendizado e comunicação.

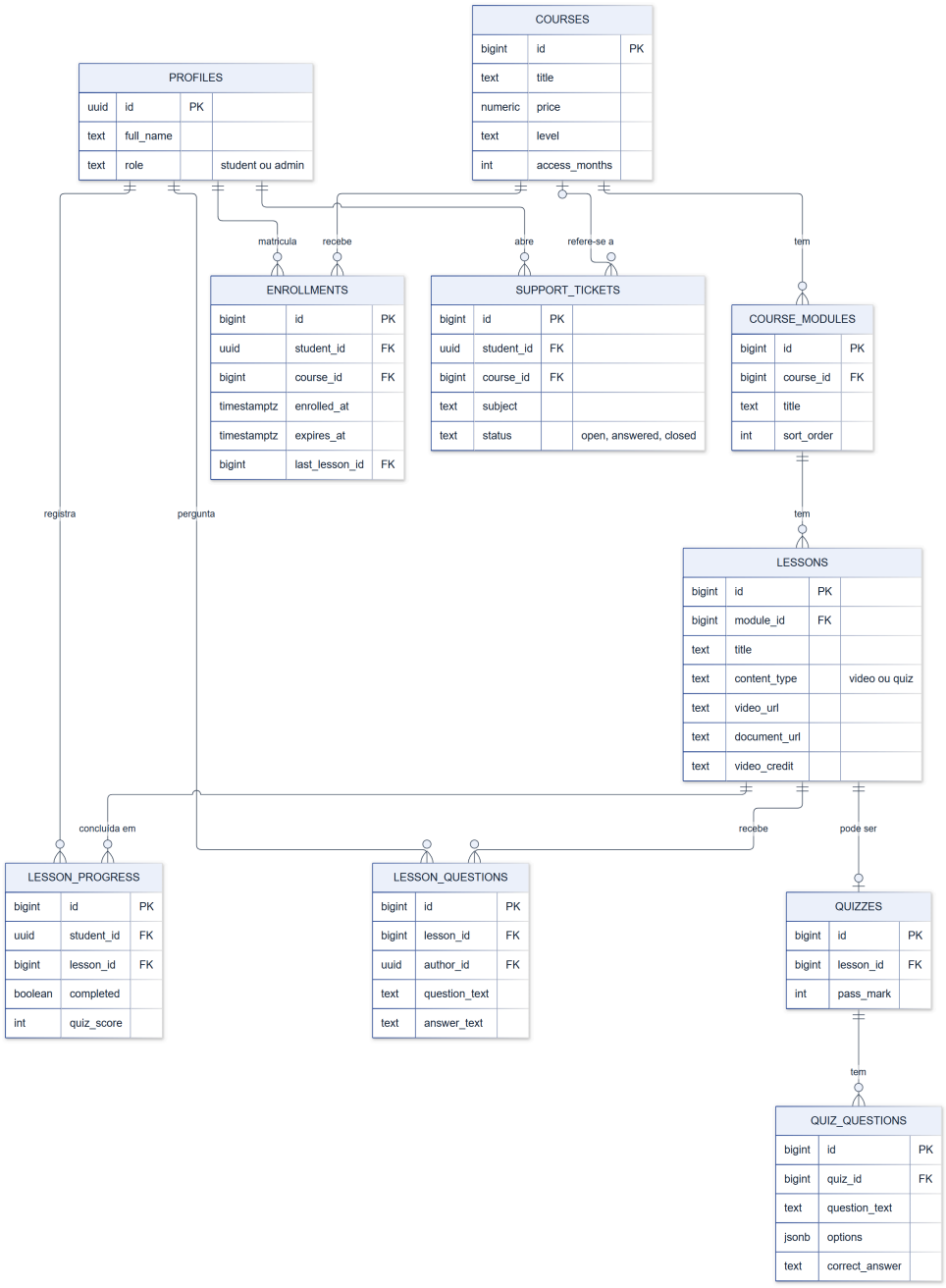


Figura 8 – Modelo entidade-relacionamento do CursoLab.

Tabela 9 – Tabelas do banco de dados.

Tabela	**Área**	**Papel**
profiles	Pessoas	Perfil de cada conta: nome e papel (aluno ou administrador). Criado por um gatilho no cadastro.
courses	Catálogo	Curso: título, preço, nível, capa e prazo de acesso em meses.
course_modules	Catálogo	Módulos de um curso, com ordem única por curso.

lessons	Catálogo	Aulas de um módulo: tipo (vídeo ou quiz), vídeo, material, descrição e crédito.
quizzes	Catálogo	Nota mínima de uma aula do tipo quiz.
quiz_questions	Catálogo	Perguntas, opções e resposta correta.
enrollments	Aprendizado	Matrícula: aluno, curso, data, prazo de acesso e última aula aberta.
lesson_progress	Aprendizado	Conclusão de cada aula por aluno e melhor nota de cada quiz.
lesson_questions	Comunicação	Dúvidas enviadas nas aulas e respostas do instrutor.
support_tickets	Comunicação	Chamados de suporte com assunto, mensagem, status e resposta.

Algumas decisões de modelagem merecem destaque:

- **A matrícula é a entidade central do aprendizado.** Ela liga aluno e curso e guarda a data de expiração do acesso e a última aula aberta, usadas pelo prazo de acesso e pelo "continue de onde parou".
- **Os módulos liberados não são armazenados.** Eles são derivados do conjunto de aulas concluídas. Guardá-los criaria um segundo estado capaz de divergir do primeiro.
- **O quiz é um tipo de aula.** Isso permite ordená-lo junto com os vídeos dentro do módulo e tratar a conclusão de forma uniforme na tabela de progresso.
- **Restrições ficam no próprio banco.** Chaves estrangeiras com exclusão em cascata, restrições de unicidade (por exemplo, um aluno só tem uma matrícula por curso e uma linha de progresso por aula) e checagens de valores (nota entre 0 e 100, status do chamado) protegem os dados mesmo que a API tenha um erro.

7.4 DIAGRAMAS DE SEQUÊNCIA

Os diagramas de sequência mostram a ordem temporal das interações. A figura a seguir apresenta o fluxo de autenticação, incluindo a renovação automática da sessão.

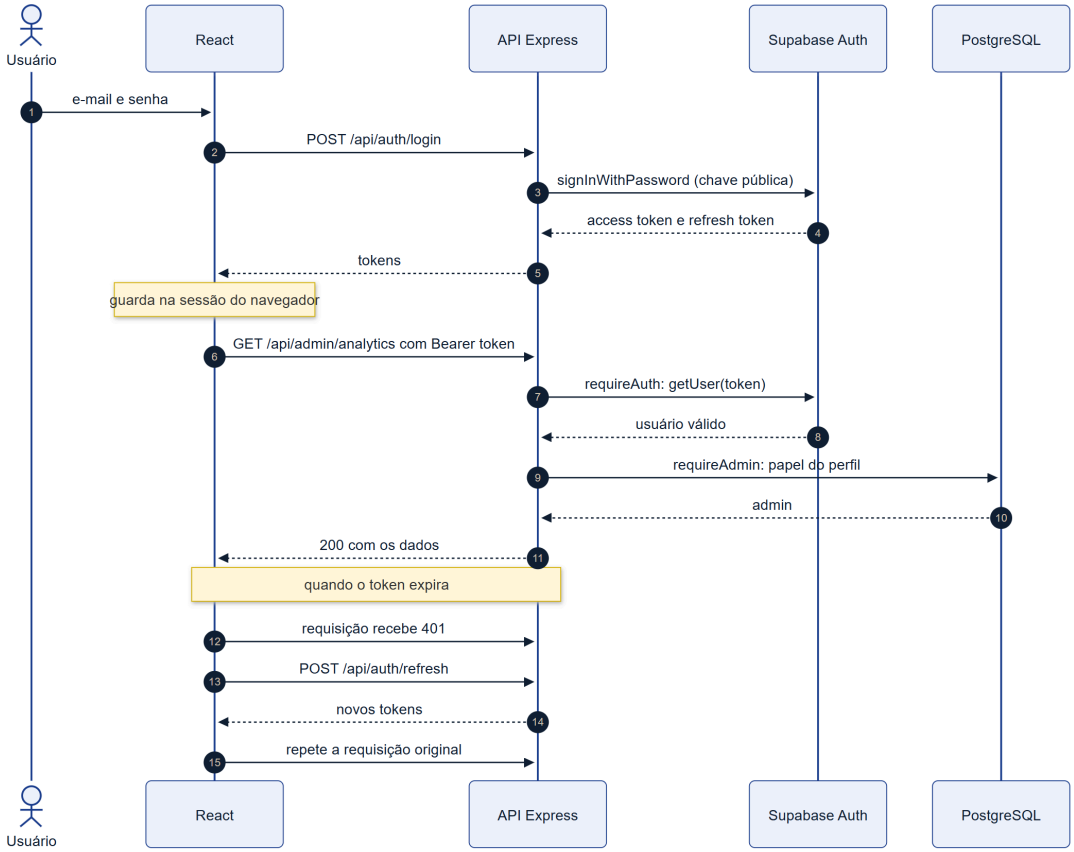


Figura 9 – Sequência de autenticação, acesso a uma rota protegida e renovação da sessão.

A figura seguinte apresenta o fluxo crítico de correção de um quiz. O ponto principal é que o gabarito é lido no banco e comparado no servidor; o navegador envia apenas as respostas e recebe apenas o resultado.

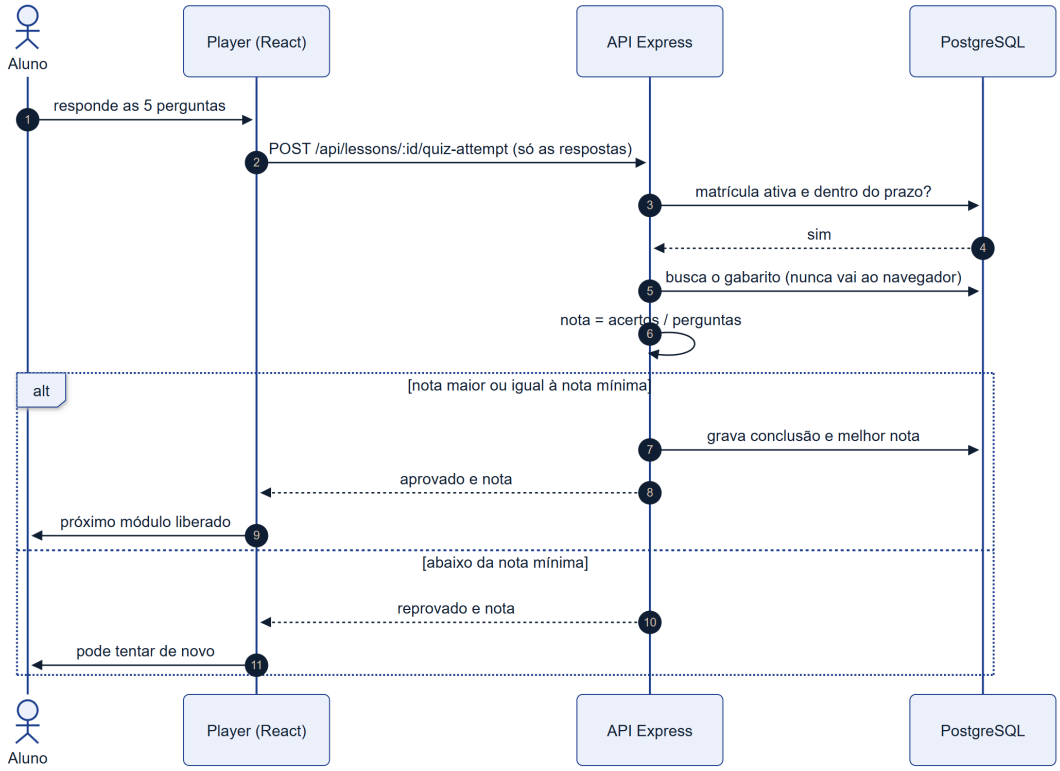


Figura 10 – Sequência da correção de um quiz.

7.5 DIAGRAMAS DE ESTADOS

Duas entidades possuem ciclo de vida relevante: a matrícula e o chamado de suporte.

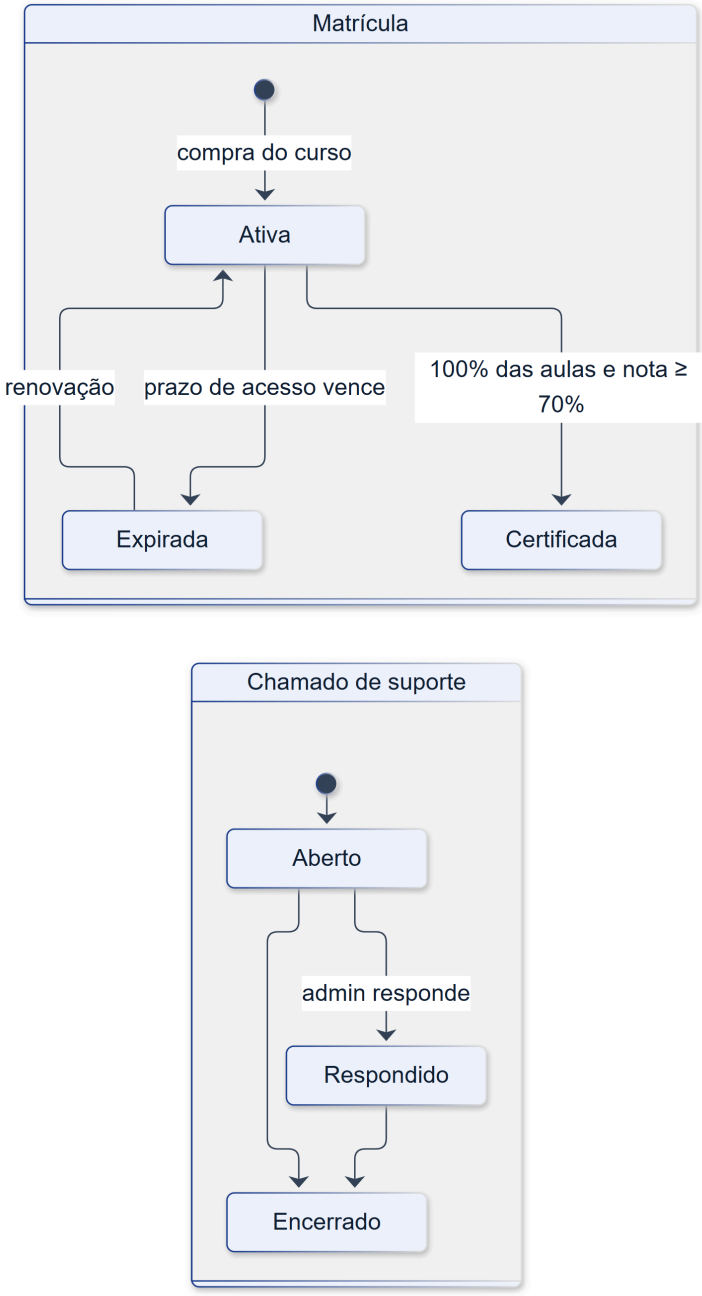


Figura 11 – Estados da matrícula e do chamado de suporte.

A matrícula começa ativa na compra, torna-se expirada quando o prazo de acesso vence e volta a ser ativa quando o aluno renova, sem perder o progresso. O certificado conquistado continua disponível mesmo depois da expiração. O chamado começa aberto, pode ser respondido pelo administrador e é encerrado pelo aluno ou pelo administrador.

8 TECNOLOGIAS UTILIZADAS

Tabela 10 – Tecnologias empregadas no projeto.

Tecnologia	**Versão**	**Finalidade**
React	19	Construção da interface em componentes.
Vite	7	Servidor de desenvolvimento e build do front-end.
Node.js	20+	Execução da API e dos scripts.
Express	5	API REST: 36 rotas e middlewares de acesso.
Supabase PostgreSQL	15+	Banco relacional: tabelas, restrições e funções.
Supabase Auth	–	Contas, hash de senha e tokens JWT.
Supabase Storage	–	Armazenamento de vídeos, materiais e apostilas.
supabase-js	2	Cliente de acesso ao Supabase a partir da API.
jsPDF	4	Geração do certificado e das apostilas em PDF.
lucide-react	1	Ícones da interface.
Vercel	–	Hospedagem, CDN, função serverless e publicação automática.
GitHub	–	Versionamento, branches e Pull Requests.
ESLint	9	Análise estática do código.

8.1 JUSTIFICATIVA DAS ESCOLHAS

8.1.1 React e Vite

O React permite construir a interface como composição de componentes reutilizáveis, o que favorece a consistência visual: botões, campos, quadros e listas seguem a mesma linguagem em todas as telas. O Vite oferece um servidor de desenvolvimento rápido e, no build, gera arquivos estáticos que a Vercel distribui pela CDN.

8.1.2 Node.js e Express

O Express é um framework simples e maduro para APIs HTTP, com o conceito de middleware, que encaixou bem no controle de acesso em camadas. Manter front-end e API na mesma linguagem (JavaScript) reduziu a quantidade de tecnologias diferentes a gerenciar.

8.1.3 Supabase

O Supabase fornece um PostgreSQL gerenciado, autenticação e armazenamento de arquivos em um único serviço. A escolha de um banco relacional foi motivada pelo domínio:

cursos têm módulos, módulos têm aulas, matrículas e progresso referenciam alunos e aulas. Essas relações e suas restrições são naturalmente expressas em SQL. O Supabase Auth evitou implementar do zero o armazenamento seguro de senhas e a emissão de tokens.

8.1.4 Vercel

A Vercel hospeda o site estático e executa a API como função serverless, com publicação automática a partir do GitHub. Essa combinação tornou o deploy contínuo uma consequência natural do fluxo de Pull Requests.

8.1.5 jsPDF

A geração de PDF no navegador evita processamento no servidor. O certificado é autorizado pela API, que confere a regra e devolve os dados, e montado no navegador. A mesma biblioteca, executada no Node.js, gera as apostilas dos cursos.

9 DESCRIÇÃO DOS MÓDULOS E REGRAS DE NEGÓCIO

Este capítulo descreve os módulos do sistema e as regras de negócio que cada um aplica. As figuras são capturas da versão publicada.

9.1 ÁREA PÚBLICA E AUTENTICAÇÃO

A página inicial apresenta a plataforma, os cursos em destaque e as perguntas frequentes, e leva ao login. Na tela de login, o usuário entra com e-mail e senha ou cria uma conta. Contas criadas pelo cadastro recebem o papel de aluno; o papel de administrador é atribuído diretamente no banco.

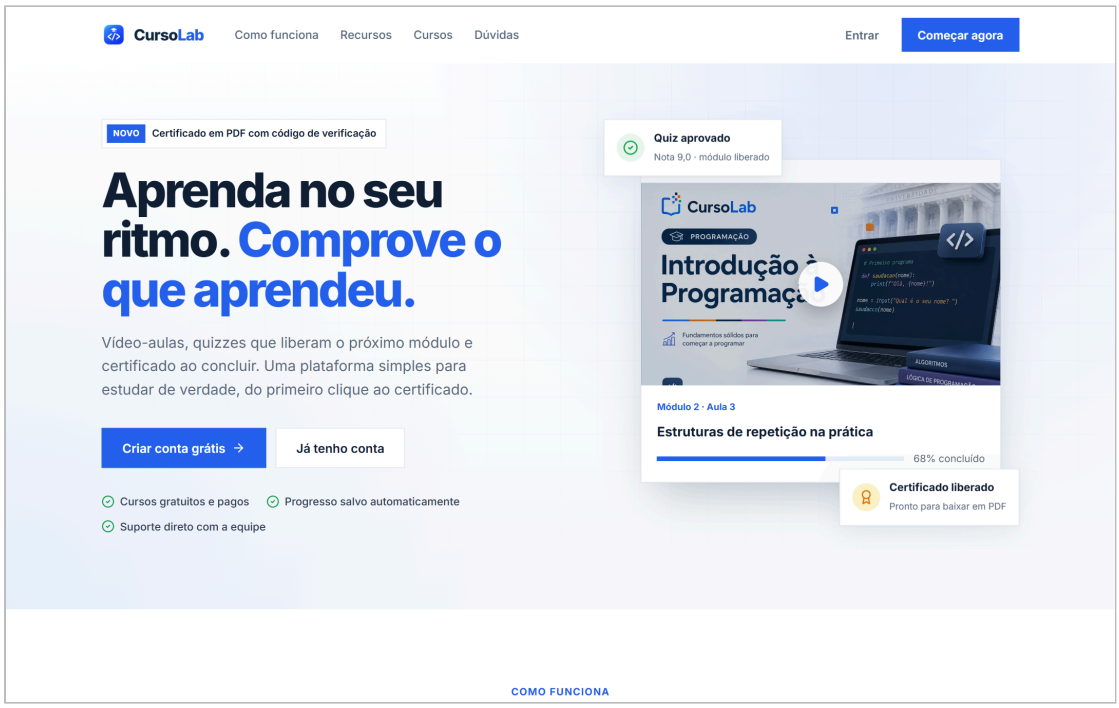


Figura 12 – Página inicial da plataforma.

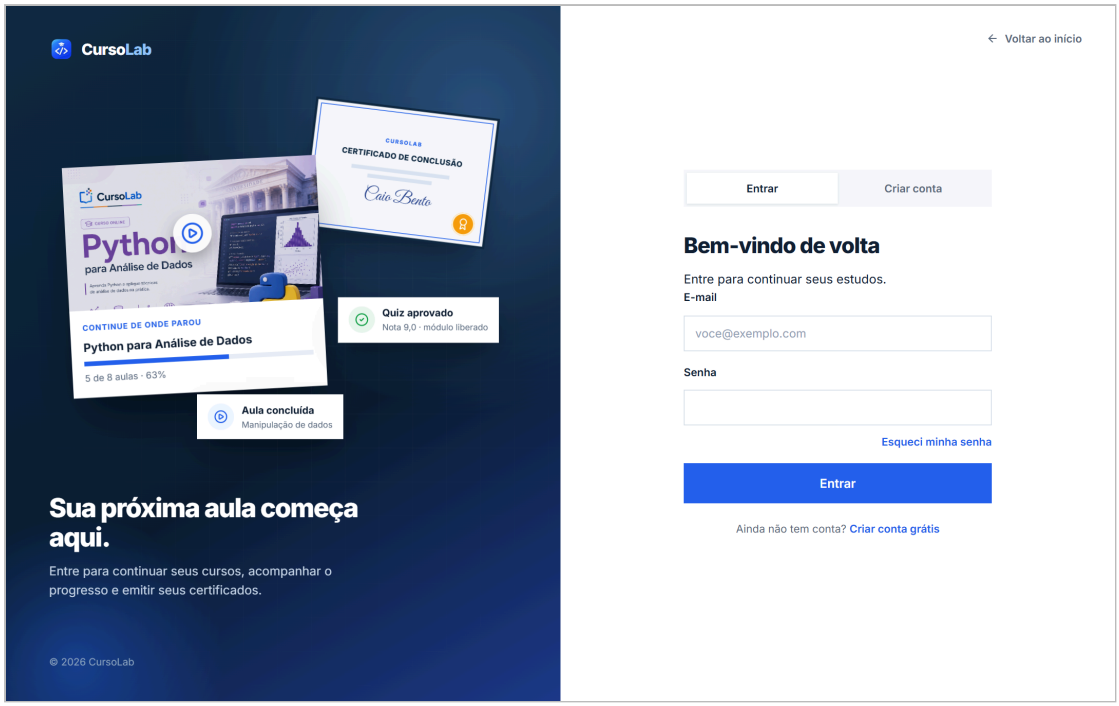


Figura 13 – Tela de login e cadastro.

O login é feito pela API, que consulta o Supabase Auth com a chave pública. Em caso de sucesso, a API devolve um token de acesso (JWT de curta duração) e um token de renovação. O front-end guarda os dois apenas na sessão do navegador e envia o token de acesso no cabeçalho `Authorization` de cada requisição. Quando a API responde com 401 porque o token expirou, o cliente da API solicita novos tokens com o token de renovação e repete a requisição original, sem que o usuário perceba. A recuperação de senha envia ao e-mail do usuário um link que abre a própria plataforma na tela de nova senha.

9.2 CATÁLOGO E DETALHES DO CURSO

O catálogo lista os cursos com busca pelo título e filtros por nível, preço e avaliação. Cursos em que o aluno já está matriculado aparecem como "Matriculado". A página de detalhes mostra a grade do curso: módulos, aulas, durações e descrições. Para quem não está matriculado, a API devolve somente a grade, sem os links dos vídeos, dos materiais nem as perguntas dos quizzes.

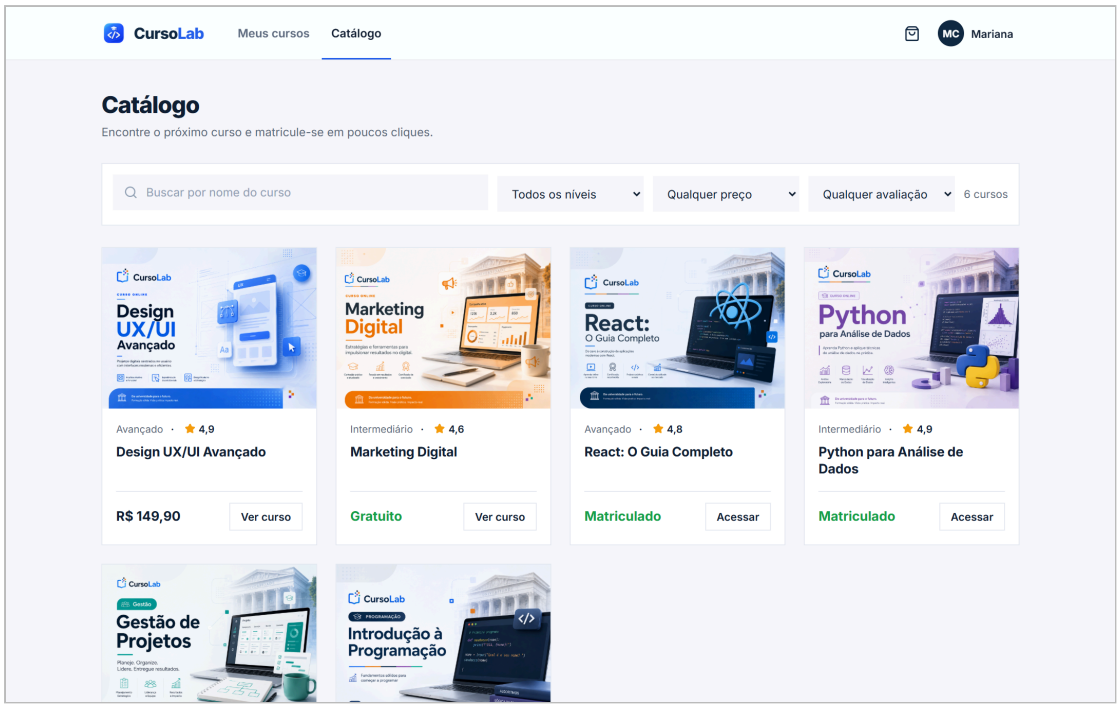


Figura 14 – Catálogo de cursos com busca e filtros.

9.3 CARRINHO, CHECKOUT E MATRÍCULA

O aluno adiciona cursos ao carrinho e conclui a compra no checkout, que é demonstrativo: aceita cartão (com validação do número no navegador) ou Pix, mas nenhum valor é cobrado e os dados do cartão não são enviados nem armazenados. A confirmação cria as matrículas.

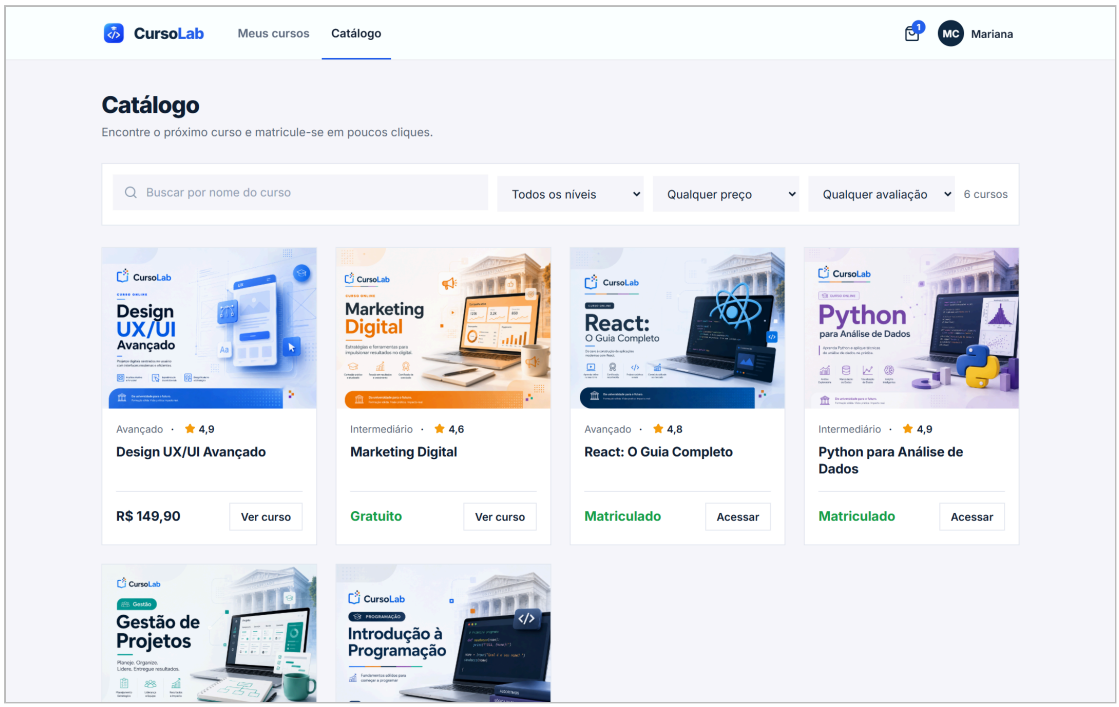


Figura 15 – Checkout demonstrativo.

Regra do prazo de acesso. Cada curso define um prazo de acesso em meses (seis por padrão). Na matrícula, a API calcula a data de expiração. Uma matrícula ativa não pode ser duplicada. Quando o prazo vence, o curso continua na lista do aluno, mas o conteúdo volta a ser apenas a grade, e progresso, dúvidas e registro de acesso são bloqueados. Comprar o curso novamente renova o prazo a partir da data atual, preservando o progresso. O certificado já conquistado continua disponível.

9.4 PAINEL DO ALUNO E "CONTINUE DE ONDE PAROU"

O painel reúne os cursos do aluno, o progresso de cada um, os certificados, os chamados de suporte e os dados da conta. O destaque do painel é o "continue de onde parou".

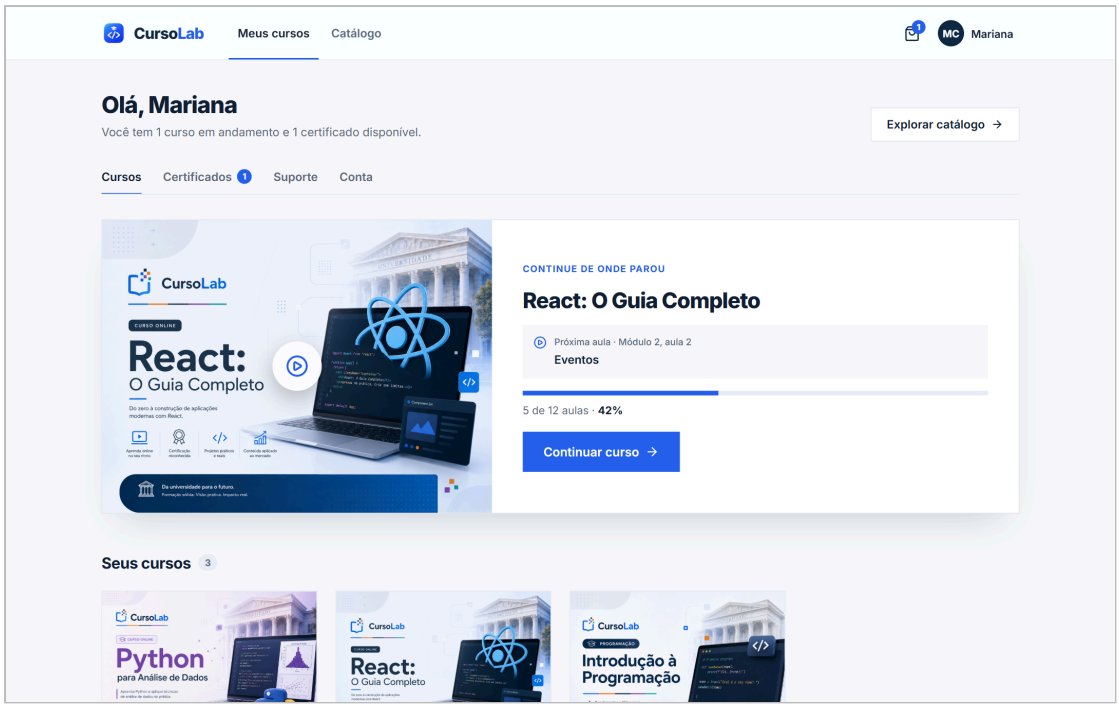


Figura 16 – Painel do aluno com o "continue de onde parou".

Regra da próxima aula. A matrícula guarda a última aula aberta pelo aluno. A próxima aula sugerida é: essa aula, se ainda não foi concluída; caso contrário, a próxima aula não concluída depois dela; e, sem acesso registrado, a primeira aula não concluída. Aulas de módulos bloqueados nunca são sugeridas: nesse caso vale a primeira pendente dos módulos liberados, que é justamente a que libera o próximo módulo. O cálculo é feito pela API e usado tanto pelo painel quanto pelo player.

9.5 PLAYER E PROGRESSÃO POR MÓDULOS

O player exibe o vídeo, o crédito ao canal de origem, a descrição da aula, o material de apoio (a apostila em PDF do módulo), a barra de progresso e a grade do curso, com os módulos bloqueados indicados por um cadeado. Também é no player que o aluno envia dúvidas sobre a aula.

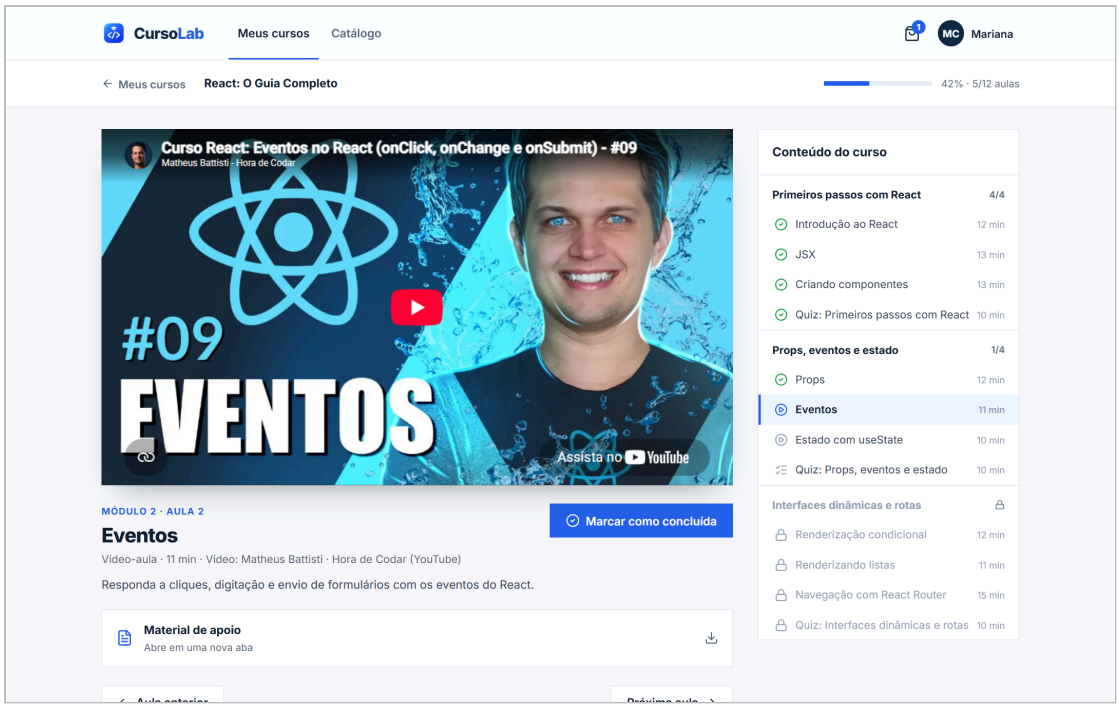


Figura 17 – Player de aulas com vídeo, crédito, descrição, material e grade do curso.

Regra de progressão. O primeiro módulo está sempre liberado. Um módulo só é liberado quando o anterior está concluído: se o módulo anterior tem quiz, o quiz precisa estar aprovado; se não tem, todas as aulas de vídeo precisam estar concluídas. Os módulos liberados não são gravados no banco: são derivados das aulas concluídas a cada consulta. A marcação de conclusão é otimista: a interface marca a aula na hora e desfaz a marcação se a API responder com erro.

9.6 QUIZZES

Cada módulo dos cursos reais termina com um quiz de cinco perguntas e nota mínima de 70%.

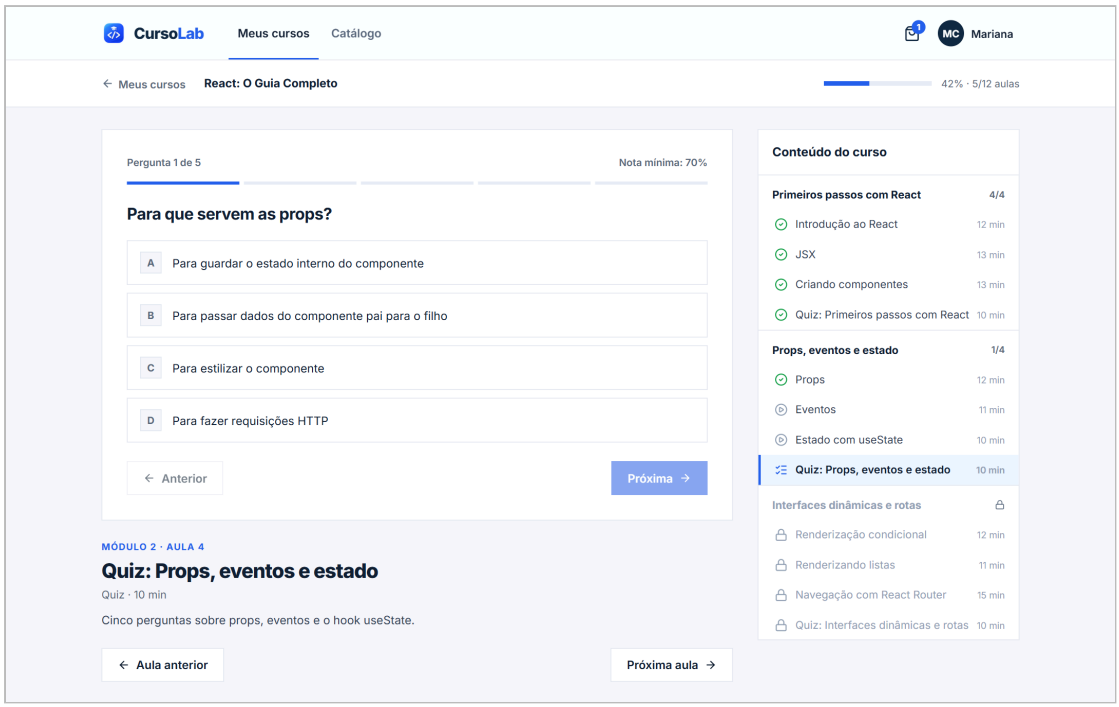


Figura 18 – Quiz no player.

Regra do quiz. O aluno envia apenas as respostas. A API confere a matrícula e o prazo, lê o gabarito no banco, calcula a nota como a proporção de acertos e compara com a nota mínima. Se aprovado, a API grava a conclusão do quiz e a melhor nota obtida; refazer o quiz pode aumentar a nota, nunca diminuí-la. A resposta da API traz a nota e o resultado, mas nunca o gabarito. Um quiz não pode ser marcado como concluído pela rota comum de progresso; só a aprovação o conclui.

9.7 CERTIFICADO

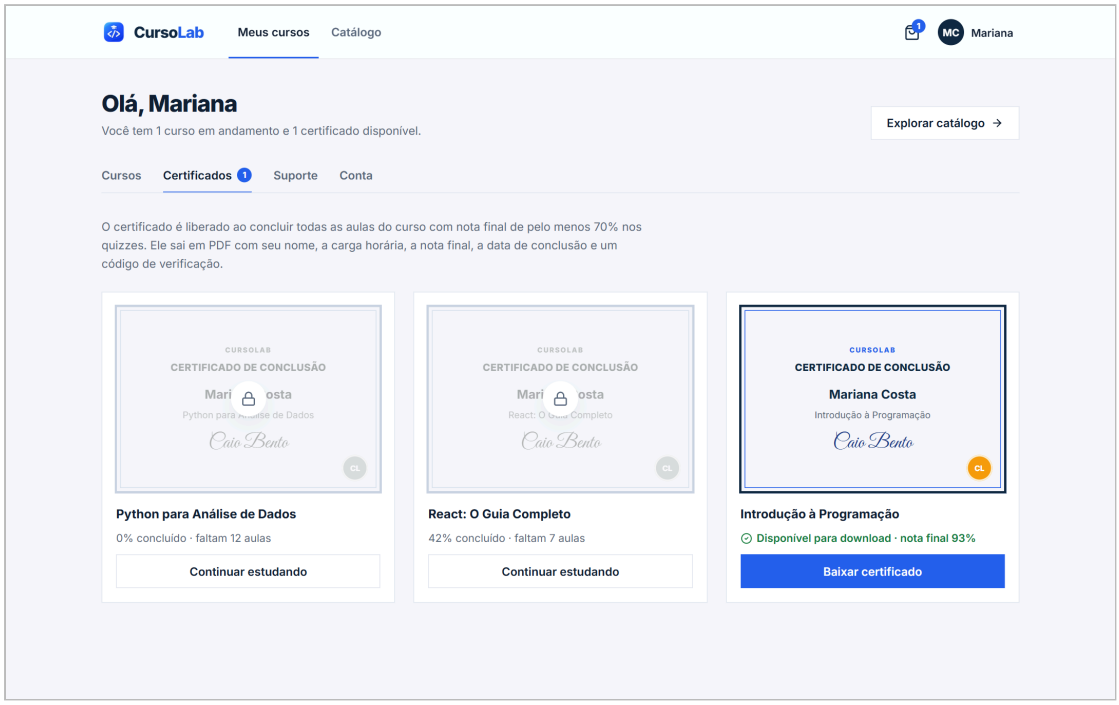


Figura 19 – Aba de certificados do painel do aluno.

Regra do certificado. O certificado é liberado quando o aluno conclui **todas as aulas do curso** e tem **nota final de pelo menos 70%**, sendo a nota final a média da melhor nota de cada quiz. A API confere a regra antes de devolver os dados e, quando o certificado não está disponível, explica o motivo (aulas pendentes ou nota insuficiente). O PDF é montado no navegador com o nome do aluno, o curso, a carga horária (soma das durações das aulas), a nota final, a data de conclusão, a assinatura do coordenador e um código de verificação derivado da matrícula.

9.8 DÚVIDAS POR AULA

O aluno com acesso ao curso pode enviar dúvidas em qualquer aula. As dúvidas aparecem para os demais alunos da aula com o nome do autor e, no backoffice, em uma fila de "sem resposta" para o instrutor. A resposta do instrutor fica visível na aula.

9.9 CHAMADOS DE SUPORTE

O aluno abre chamados gerais ou sobre um curso em que está matriculado e acompanha o status: aberto, respondido ou encerrado. O administrador responde ou encerra pelo backoffice. Um aluno com acesso expirado ainda pode abrir chamado sobre o curso, pois o suporte é justamente o canal para resolver esse tipo de situação.

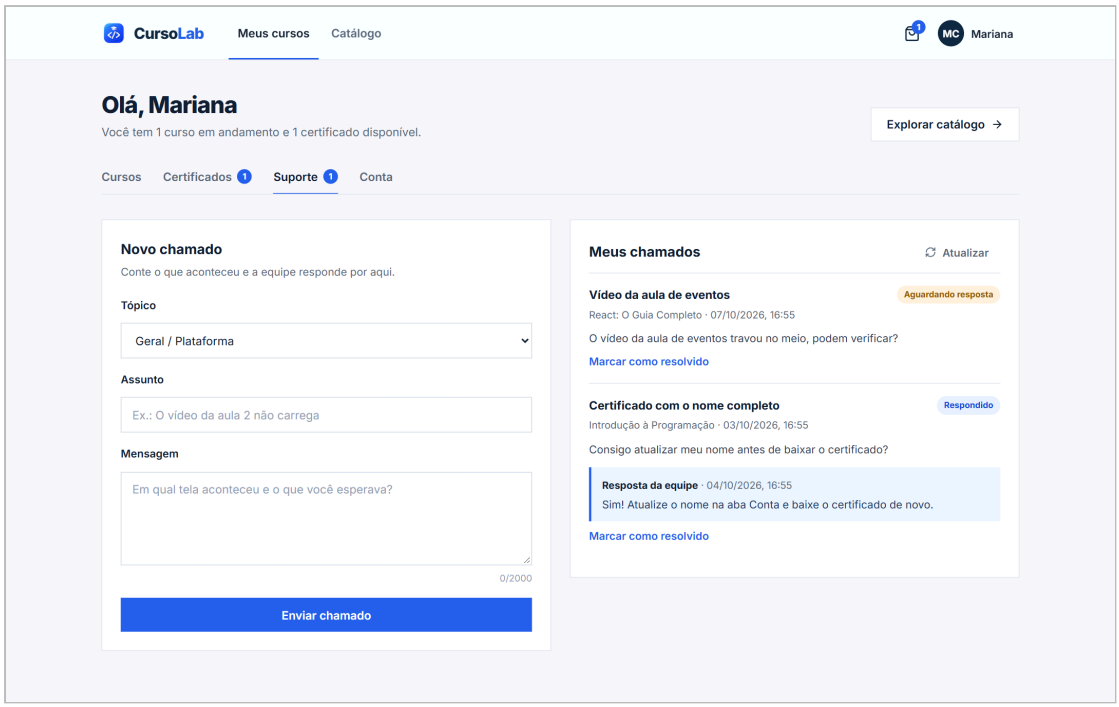


Figura 20 – Chamados de suporte na área do aluno.

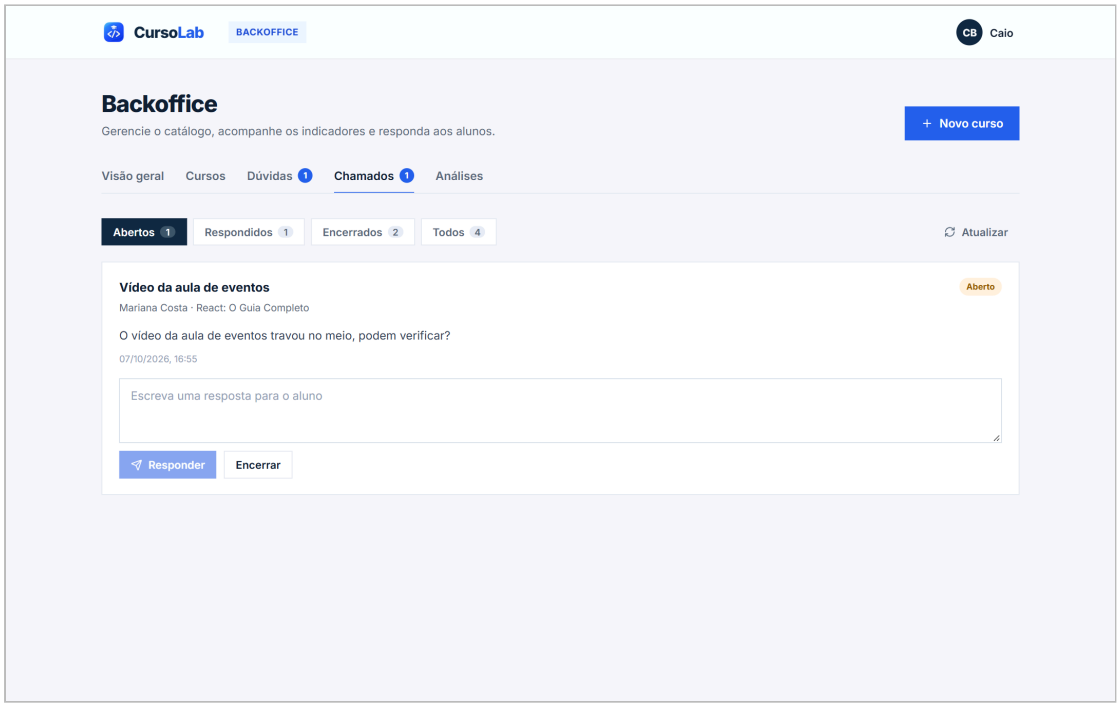


Figura 21 – Fila de chamados no backoffice.

9.10 BACKOFFICE E EDITOR DE CONTEÚDO

O backoffice é restrito ao papel de administrador. A visão geral mostra indicadores e pendências (dúvidas sem resposta e chamados abertos). O instrutor cria cursos informando título, preço, nível e prazo de acesso, e monta o conteúdo no editor: módulos, aulas de vídeo

(com envio de arquivo ou link do YouTube ou Vimeo), descrição, crédito do vídeo, material de apoio e quizzes com perguntas e nota mínima.

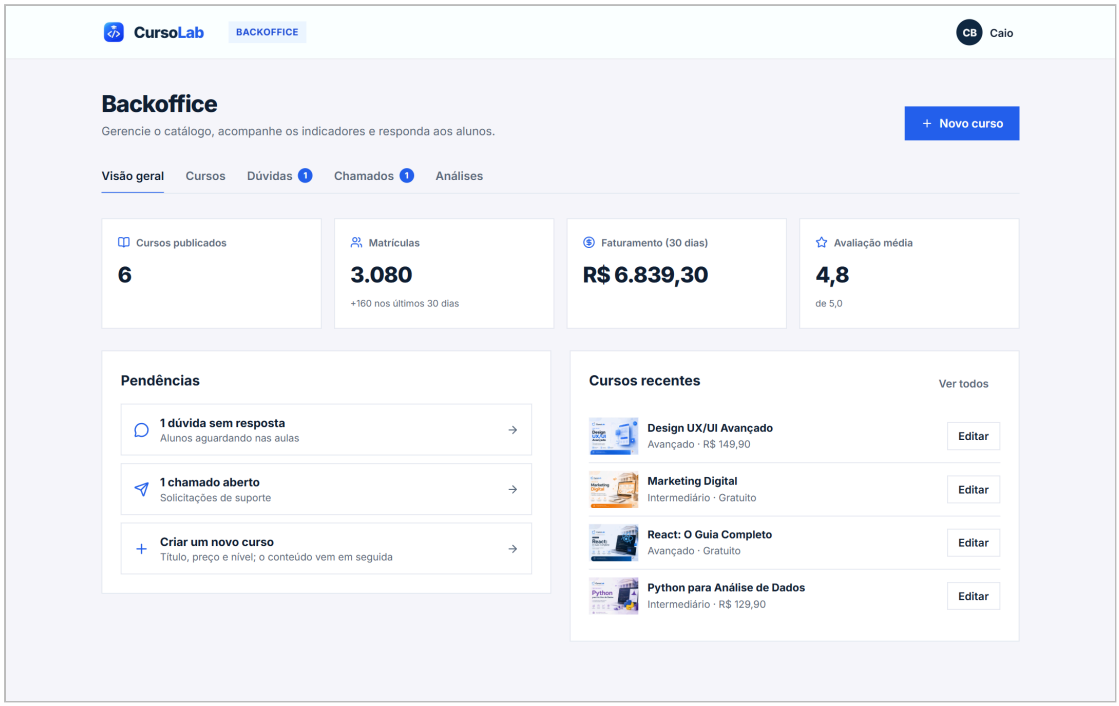


Figura 22 – Visão geral do backoffice.

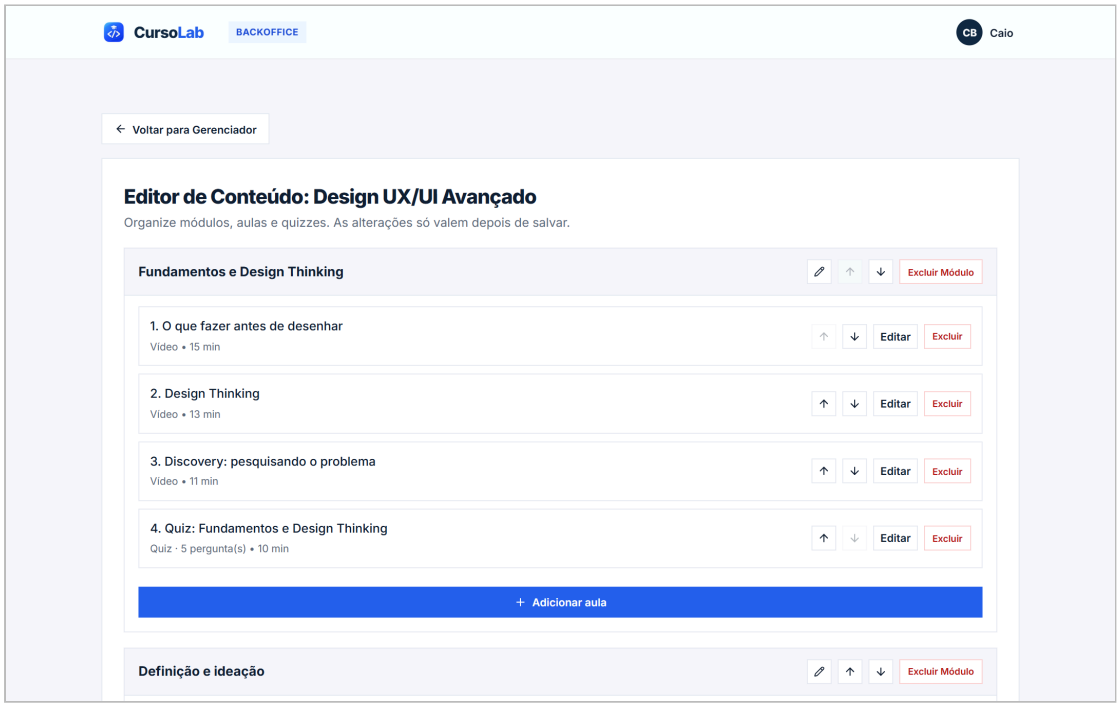


Figura 23 – Editor de conteúdo do curso.

Regra de gravação do conteúdo. O editor permite várias alterações (criar, renomear, reordenar e excluir módulos e aulas) e as envia de uma vez. A API valida todo o conteúdo e chama a função `save_course_content` do banco, que grava tudo numa única transação. Aulas

existentes são atualizadas no lugar, e não recriadas, para não apagar o progresso e as dúvidas dos alunos. Se qualquer parte falhar, nada é alterado.

9.11 INDICADORES

A aba de análises apresenta, por curso, matrículas, novos alunos e faturamento dos últimos 30 dias e avaliação. Os indicadores somam números de demonstração, gravados no catálogo para dar volume à apresentação, às matrículas reais calculadas a partir da tabela de matrículas. Essa mistura é uma limitação conhecida, registrada no Capítulo 19.

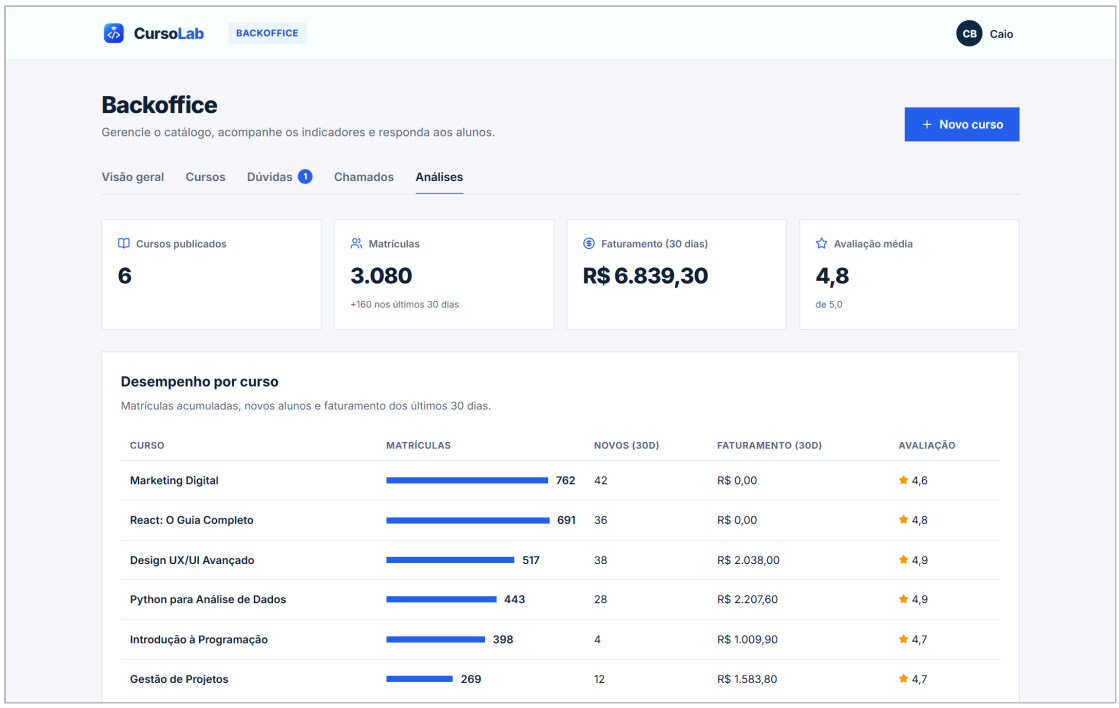


Figura 24 – Indicadores por curso no backoffice.

9.12 CONTEÚDO REAL DOS CURSOS

Os seis cursos do catálogo têm conteúdo real, cadastrado por um script versionado (`npm run seed:cursos`). Cada curso tem três módulos, e cada módulo tem três vídeos de canais educacionais do YouTube, verificados como públicos e incorporáveis, um quiz de cinco perguntas e uma apostila em PDF escrita para a plataforma e enviada ao Storage. Os vídeos são exibidos com o player oficial do YouTube, no modo de privacidade aprimorada, e com o crédito ao canal na aula e na apostila. O script pode ser executado novamente sem perda de dados: aulas com o mesmo título mantêm o identificador e, portanto, o progresso dos alunos.

10 SEGURANÇA E CONSISTÊNCIA

10.1 AUTENTICAÇÃO E SESSÃO

As senhas são armazenadas pelo Supabase Auth apenas como hash; o código do projeto nunca grava nem lê senhas. A sessão é representada por um token JWT assinado e de curta duração, renovado com um token de renovação. Os tokens ficam na sessão do navegador (`sessionStorage`) e são descartados ao fechar a aba ou sair. Cada rota protegida valida o token no Supabase Auth por meio do middleware `requireAuth`.

10.2 AUTORIZAÇÃO

Autenticar diz quem é o usuário; autorizar diz o que ele pode fazer. O CursoLab aplica a autorização em três níveis:

- **por papel:** o middleware `requireAdmin` libera as operações de gestão apenas para perfis com papel de administrador;
- **por recurso:** vídeos, materiais, perguntas dos quizzes, progresso e dúvidas exigem matrícula dentro do prazo; chamados exigem que o aluno seja o autor;
- **por campo:** o mesmo conteúdo de curso é entregue em três versões. O visitante recebe só a grade, o aluno matriculado recebe o conteúdo sem o gabarito e o administrador recebe tudo, para poder editar.

Toda essa verificação acontece no servidor. Esconder um botão na interface não é controle de acesso, pois qualquer requisição pode ser enviada diretamente à API.

10.3 SEPARAÇÃO DE SEGREDOS

O Supabase fornece duas chaves. A **chave pública** é usada apenas para o login. A **chave secreta**, que tem acesso total ao banco, existe apenas no servidor, como variável de ambiente local (fora do repositório) e na Vercel; ela nunca é enviada ao navegador. Além disso, as funções do banco que fazem operações administrativas tiveram a permissão de execução revogada para os papéis públicos do Supabase, de modo que só o backend consegue executá-las.

10.4 VALIDAÇÃO DE ENTRADA

Toda entrada é validada na API antes de chegar ao banco: campos obrigatórios, tamanhos máximos (por exemplo, descrição da aula com até 1000 caracteres), URLs que

precisam começar com `http://` ou `https://`, notas entre 0 e 100, respostas de quiz que precisam estar entre as opções, tipos e tamanhos de arquivo permitidos. Mensagens de erro claras são devolvidas com o código HTTP adequado (400, 401, 403, 404 ou 409). A API também aceita requisições apenas da origem configurada (CORS).

10.5 TRANSAÇÕES E RESTRIÇÕES

A consistência é garantida em duas camadas. Na API, operações compostas são delegadas a funções do banco que executam numa única transação, como a gravação do conteúdo do curso. No banco, restrições de integridade impedem dados inválidos mesmo em caso de erro na aplicação: chaves estrangeiras, unicidade (uma matrícula por aluno e curso, uma ordem por módulo), checagens de valores e exclusão em cascata. As migrações foram escritas para poder ser executadas mais de uma vez sem erro.

11 VERIFICAÇÃO, VALIDAÇÃO E TESTES

11.1 VERIFICAÇÃO E VALIDAÇÃO

Verificação avalia se o produto foi construído corretamente, isto é, conforme a especificação. **Validação** avalia se o produto correto foi construído, isto é, se ele atende à necessidade. No CursoLab, a verificação foi feita por análise estática, build e testes automatizados; a validação, pelo uso do sistema publicado como visitante, aluno e administrador, com conteúdo real. A mudança da regra do certificado mostra a diferença: a regra de 60% estava corretamente implementada (verificação), mas não era a regra adequada (validação).

11.2 ANÁLISE ESTÁTICA E DINÂMICA

A análise estática é feita pelo ESLint, executado antes de cada Pull Request, e pelo build do Vite, que falha diante de erros de sintaxe ou importações inválidas. A revisão do próprio diff de cada Pull Request também é uma forma de análise estática. A análise dinâmica é feita pela suíte de testes de integração, que executa a API de verdade, e pela navegação no sistema publicado.

11.3 TESTES AUTOMATIZADOS

O projeto possui uma suíte de **testes de integração automatizados** em `backend/tests`, executada com `npm run test:api`. Os testes chamam a API real e usam o banco configurado. Cada suíte cria seus próprios usuários, com e-mails `@cursolab.test`, e cursos temporários, e apaga tudo ao final. A mesma suíte pode ser executada contra a API local ou contra a produção, passando o endereço como parâmetro.

Tabela 11 – Suítes de testes automatizados.

Suíte	Verificações	O que cobre
conteudo	23	gravação do conteúdo em transação, reordenação, validações e payloads inválidos
seguranca-do-quiz	22	grade sem gabarito, correção no servidor, melhor nota e regra do certificado
painel-aluno	25	progresso, próxima aula, certificado, carga horária, nome e troca de senha
prazo-de-acesso	18	expiração, bloqueio do conteúdo, renovação e manutenção do progresso

continuar-de-onde-parou	16	cálculo da próxima aula com módulos bloqueados e acesso registrado
chamados	22	abertura, resposta, encerramento e permissões dos chamados
duvidas	15	envio, resposta e permissões das dúvidas por aula
uploads	17	URL assinada, envio ao Storage, tipos e tamanhos permitidos
Total	**158**	

Os testes priorizam os requisitos inversos, isto é, o que não pode acontecer: visitante recebendo vídeo ou gabarito, aluno sem matrícula acessando aula, quiz marcado sem ser feito, certificado com nota insuficiente, dado inválido alterando o banco e acesso expirado liberando conteúdo. Todas as suítes passaram localmente e contra a produção após o último merge.

11.4 NÍVEIS DE TESTE

Tabela 12 – Níveis de teste e aplicação ao CursoLab.

Nível	**Objetivo**	**Aplicação no projeto**
Unitário	Verificar pequenas unidades isoladamente.	Não há testes unitários formais; as regras são cobertas pela integração. Evolução futura.
Integração	Verificar a interação entre componentes.	8 suítes que exercitam API, Supabase Auth, PostgreSQL e Storage juntos.
Sistema	Verificar a aplicação completa.	Suítes executadas contra a produção e navegação no sistema publicado.
Aceitação	Confirmar a adequação ao uso.	Roteiro de demonstração com os fluxos do aluno e do administrador.
Regressão	Garantir que mudanças não quebrem o que existia.	A suíte completa é executada antes de cada Pull Request e depois de cada merge.

11.5 TESTES FUNCIONAIS E NÃO FUNCIONAIS

Os testes automatizados são testes funcionais de **caixa-preta** e de **API**: enviam requisições e verificam o código de resposta e o conteúdo, sem depender da implementação interna. Também funcionam como testes de **regressão** e de **fumaça** após cada publicação.

Os testes não funcionais não foram todos formalizados. Há verificações de segurança, como acesso sem token, acesso de outro usuário e exposição do gabarito, mas não foram executados testes de carga, de estresse ou de desempenho com métricas definidas. Esses testes são registrados como evolução.

11.6 MODELO V

O Modelo V relaciona cada etapa de construção à atividade de verificação ou validação correspondente.

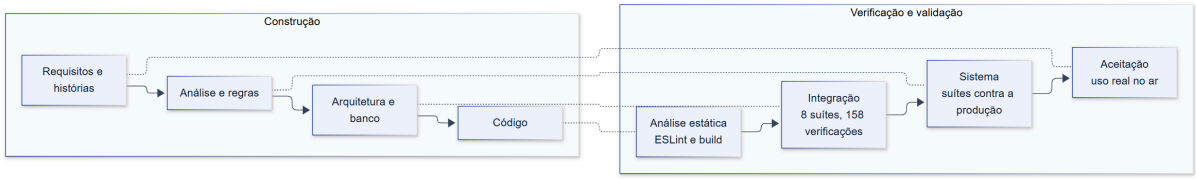


Figura 25 – Relação entre etapas de construção e níveis de teste no Modelo V.

12 MANUTENÇÃO E EVOLUÇÃO

A manutenção de software reúne as atividades que mantêm o sistema funcional, seguro e adequado depois de entregue. Como o CursoLab esteve publicado durante todo o desenvolvimento, houve exemplos reais dos quatro tipos de manutenção.

12.1 MANUTENÇÃO CORRETIVA

A rota que registra a última aula aberta devolvia "acesso expirado" (403) para um aluno que nem sequer era matriculado no curso. A correção passou a verificar primeiro a matrícula (404) e depois o prazo (403), e foi acompanhada de testes que impedem a volta do erro. Toda correção seguiu o mesmo princípio: reproduzir o defeito, corrigir e acrescentar uma verificação de regressão à suíte.

12.2 MANUTENÇÃO EVOLUTIVA

O prazo de acesso de seis meses, o conteúdo real dos cursos (vídeos, quizzes e apostilas), a descrição e o crédito das aulas e a nova regra do certificado foram evoluções introduzidas depois que o sistema já estava publicado, cada uma com migração, código, testes e documentação.

12.3 MANUTENÇÃO ADAPTATIVA

Para publicar na Vercel, a API em Express, originalmente um servidor que escuta uma porta, foi adaptada: o aplicativo passou a ser exportado por um módulo e reaproveitado tanto pelo servidor local quanto pela função serverless. Também foi necessário configurar as URLs de redirecionamento do Supabase para o domínio de produção, para que os links de recuperação de senha funcionassem.

12.4 MANUTENÇÃO PREVENTIVA

A suíte de testes de regressão, executada a cada mudança, é a principal medida preventiva. As migrações idempotentes, a revisão de dependências e a atualização da documentação a cada Pull Request também reduzem a chance de falhas futuras.

12.5 REFATORAÇÃO E DÍVIDA TÉCNICA

Refatorar é reestruturar o código sem alterar o comportamento externo, e os testes automatizados foram o que tornou as refatorações seguras: depois de cada mudança interna, a

suíte confirmava que o comportamento continuava o mesmo. A dívida técnica conhecida está registrada de forma explícita:

- a API está concentrada em um único arquivo, que cresceu com o projeto e poderia ser dividido por módulo (cursos, aprendizado, suporte);
- não há testes unitários nem testes automatizados da interface;
- os indicadores misturam números de demonstração com dados reais;
- os testes ainda são executados manualmente, e não automaticamente em cada Pull Request.

12.6 VERSIONAMENTO

Todo o código, as migrações do banco, o script de carga dos cursos, a documentação e as fontes deste relatório estão versionados no Git. Cada mudança pode ser associada a um Pull Request com a justificativa, o que complementa a rastreabilidade dos requisitos com a rastreabilidade técnica.

13 DOCUMENTAÇÃO DE SOFTWARE

A documentação acompanhou o código ao longo do projeto, e não foi produzida apenas no final. Cada Pull Request que alterava uma regra também atualizava os documentos afetados.

13.1 DOCUMENTAÇÃO TÉCNICA

- `README.md`: visão geral, arquitetura, funcionalidades, ordem de execução das migrações, variáveis de ambiente, testes, implantação e fluxo de desenvolvimento;
- `backend/README.md`: estrutura da API e referência das 36 rotas, com método, caminho e nível de acesso;
- `backend/database/*.sql`: migrações numeradas e comentadas, explicando o motivo de cada mudança;
- `docs/documentacao-requisitos.md`: histórias de usuário, requisitos, arquitetura, modelo de dados e decisões de projeto;
- comentários no código apenas onde a decisão não é evidente, como a regra da próxima aula, a faixa temporária de ordenação e a correção do quiz no servidor;
- este relatório e os diagramas, cujas fontes estão em `docs/entrega`.

13.2 DOCUMENTAÇÃO DE API

As rotas estão documentadas em tabela no `backend/README.md`, com método, caminho e acesso exigido. Em uma evolução, essa documentação poderia ser formalizada em OpenAPI, o que permitiria gerar uma interface de consulta e validar contratos automaticamente.

13.3 GUIA DE IMPLANTAÇÃO E INSTALAÇÃO

O README descreve a preparação completa do ambiente: execução das migrações no Supabase, criação do arquivo de variáveis de ambiente, instalação das dependências, execução local da API e do front-end, carga dos cursos reais, execução dos testes e publicação na Vercel.

13.4 DOCUMENTAÇÃO DO USUÁRIO

A interface foi escrita para dispensar manual: as telas explicam as regras no próprio contexto. O painel informa quantas aulas faltam ou qual nota falta para o certificado, o player

indica módulos bloqueados e a página inicial tem uma seção de perguntas frequentes. Um manual do usuário formal é uma possibilidade de evolução.

14 PADRÕES DE DESENVOLVIMENTO

Padrões de projeto são soluções conhecidas para problemas recorrentes. Este capítulo distingue os padrões que existem de fato no código das possibilidades de evolução, sem afirmar implementações que não existem.

14.1 PADRÕES CRIACIONAIS

Singleton (por módulo). Os clientes do Supabase (`supabaseAdmin` e `supabaseAuth`) são criados uma única vez no módulo `backend/src/supabase.js` e importados por toda a API. Como o sistema de módulos do JavaScript executa cada módulo uma única vez, há na prática uma instância única de cada cliente, sem a implementação clássica com construtor privado.

Factory Method. Não foi necessário. Seria útil, por exemplo, para criar diferentes tipos de notificação em uma evolução.

14.2 PADRÕES ESTRUTURAIS

Adapter. A função `getVideoEmbedUrl` adapta formatos de vídeo incompatíveis (links do YouTube em diferentes formatos, links do Vimeo e arquivos enviados) a uma única interface usada pelo player. O player não precisa saber de onde o vídeo vem.

Facade. O módulo `services/api.js` funciona como uma fachada para a comunicação com o backend: as telas chamam funções como `getCertificate(courseId)` sem conhecer cabeçalhos, tokens, renovação de sessão ou tratamento de erros.

14.3 PADRÕES COMPORTAMENTAIS

Chain of Responsibility. Os middlewares do Express formam uma cadeia: a requisição passa por `requireAuth` e depois por `requireAdmin`, e cada elo pode interrompê-la (401 ou 403) ou passá-la adiante com `next()`.

Strategy. Não implementado. A forma de calcular a nota final ou a regra de liberação do certificado poderiam ser encapsuladas como estratégias, caso a plataforma passasse a ter regras diferentes por curso.

14.4 PADRÃO ARQUITETURAL

O CursoLab não segue um MVC clássico com pastas separadas para model, view e controller, mas tem uma separação equivalente: o banco e as funções SQL fazem o papel do

modelo, os componentes React o da visão e as rotas da API o do controlador. A definição mais precisa para o projeto é **aplicação monolítica modularizada organizada em camadas**.

14.5 DEVSECOPS

O projeto tem práticas compatíveis com DevSecOps: segredos fora do repositório, publicação automática a cada merge, testes de segurança na suíte de integração e execução dos testes contra a produção. Ainda não há um pipeline de integração contínua que execute os testes e a análise de dependências automaticamente em cada Pull Request; esse é o próximo passo natural.

15 REUSO DE SOFTWARE

Reuso é aproveitar conhecimento, componentes ou serviços existentes em vez de reconstruí-los. No CursoLab, o reuso foi principalmente **sistemático** e **horizontal**: componentes genéricos, úteis em qualquer domínio, foram escolhidos de forma planejada.

- **React** e **Vite**: estrutura da interface e build;
- **Express**: servidor HTTP e cadeia de middlewares;
- **Supabase**: banco, autenticação e armazenamento como serviço, o que evitou implementar hash de senha, emissão de tokens e armazenamento de arquivos;
- **jsPDF**: geração de PDF do certificado e das apostilas;
- **lucide-react**: ícones;
- **vídeos do YouTube**: conteúdo educacional de terceiros reutilizado com o player oficial e crédito aos autores.

Também houve **reuso interno**: uma linguagem visual única, definida em um arquivo de estilos compartilhado, é usada por todas as telas, e o mesmo aplicativo Express é reaproveitado pelo servidor local e pela função serverless.

O reuso trouxe produtividade e confiabilidade, mas também dependências. Mudanças nas bibliotecas ou nos serviços externos podem exigir manutenção adaptativa, e a plataforma depende da disponibilidade do Supabase, da Vercel e do YouTube. Esses riscos foram aceitos conscientemente em troca de concentrar o esforço nas regras do domínio.

16 IMPLANTAÇÃO E AMBIENTE DE EXECUÇÃO

16.1 PRODUÇÃO

O sistema está publicado na Vercel. O front-end é um site estático servido pela CDN, e a API roda como função serverless no mesmo domínio, o que dispensa configuração de CORS entre domínios diferentes. O arquivo `vercel.json` redireciona as rotas `/api/*` para a função. As variáveis de ambiente (URL do Supabase, chave pública e chave secreta) são configuradas no painel da Vercel. Cada merge na branch `main` gera uma nova publicação automaticamente.

16.2 EXECUÇÃO LOCAL

Para executar localmente, são necessários Node.js 20 ou superior e um projeto no Supabase. O fluxo é:

1. executar as migrações de `backend/database` no editor SQL do Supabase, na ordem indicada no README;
2. criar `backend/.env` com a URL do projeto e as chaves;
3. instalar as dependências com `npm install`;
4. iniciar a API com `npm run server` e o front-end com `npm run dev`;
5. opcionalmente, cadastrar os cursos reais com `npm run seed:cursos`;
6. executar os testes com `npm run test:api`.

Em desenvolvimento, o Vite encaminha `/api` para a API local, de modo que o front-end usa os mesmos caminhos relativos que em produção.

16.3 DADOS DE DEMONSTRAÇÃO

O catálogo traz seis cursos com conteúdo real e indicadores de demonstração (avaliações, matrículas e faturamento) para que painéis e gráficos não apareçam vazios na apresentação. Os testes automatizados não dependem desses dados: cada suíte cria e apaga os próprios registros.

17 INTERFACE E USABILIDADE

A interface segue uma linguagem visual única e simples, definida em um arquivo de estilos compartilhado: tipografia Inter, paleta azul, quinas retas, espaçamentos consistentes e componentes reutilizados (botões, campos, quadros, abas e listas). A consistência reduz o esforço de aprendizado: o que o usuário aprende em uma tela vale para as outras.

A navegação segue o fluxo natural do usuário. O visitante conhece a plataforma pela página inicial e entra pelo login; o aluno chega ao painel, com o "continue de onde parou" em destaque, e dali ao catálogo e ao player; o administrador chega direto ao backoffice. Informações importantes aparecem no contexto em que são necessárias: o player mostra o progresso e os módulos bloqueados; o painel informa o prazo de acesso e o que falta para o certificado; erros de validação aparecem junto ao campo.

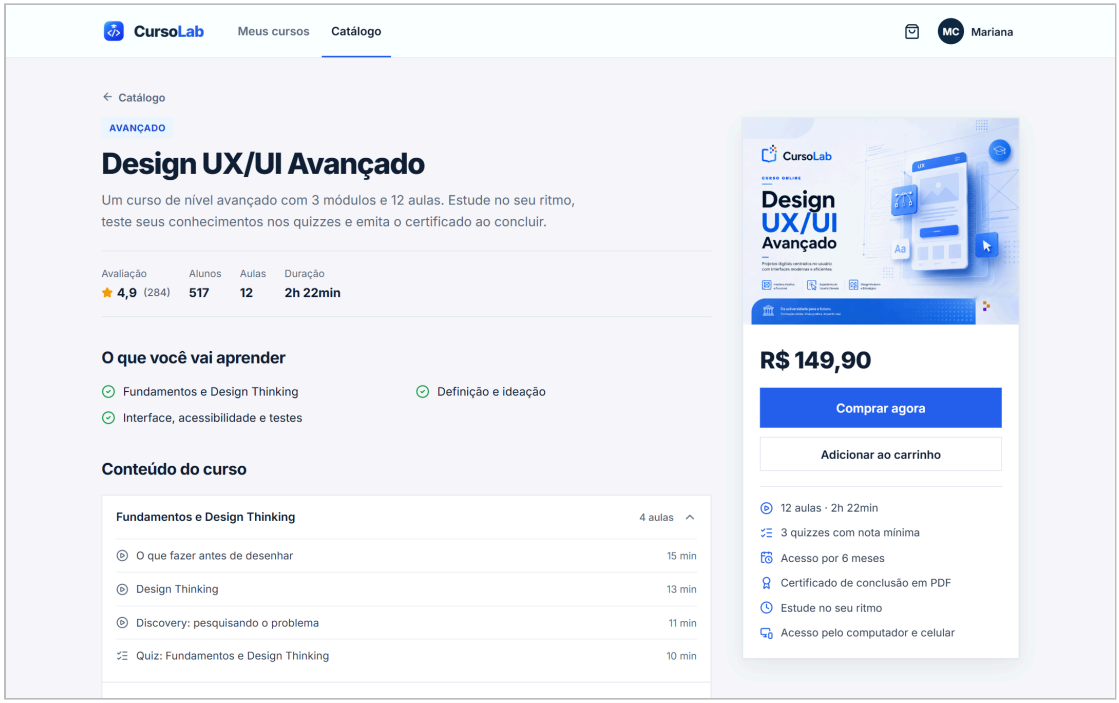


Figura 26 – Página de detalhes do curso com a grade.

A interface é responsiva e se adapta do celular ao computador. A acessibilidade foi considerada no uso de rótulos nos campos, textos alternativos nos ícones e contraste adequado, embora não tenha sido feita uma avaliação formal de acessibilidade.

18 DIFICULDADES, DECISÕES E APRENDIZADOS

18.1 ENVIO DE ARQUIVOS GRANDES

A Vercel limita o corpo de cada requisição a cerca de 4,5 MB, e o projeto precisava aceitar vídeos de até 50 MB. Enviar o arquivo pela API era inviável. A solução foi inverter o fluxo: a API apenas valida o tipo e o tamanho e gera uma **URL assinada** de envio no Storage; o navegador envia o arquivo diretamente para lá. Além de contornar o limite, isso tirou da função serverless um trabalho pesado.

18.2 TRANSAÇÃO SEM SUPORTE NO CLIENTE

Salvar o conteúdo de um curso altera quatro tabelas: módulos, aulas, quizzes e perguntas. O cliente do Supabase não oferece transações entre várias chamadas, então uma falha no meio deixaria o curso pela metade. A solução foi escrever a função `save_course_content` no próprio PostgreSQL, que executa toda a gravação numa única transação e é chamada pela API como um procedimento remoto. A permissão de execução dessa função foi revogada para os papéis públicos, de modo que só o backend pode chamá-la.

18.3 REORDENAÇÃO COM RESTRIÇÃO DE UNICIDADE

A ordem dos módulos e das aulas é única dentro de cada curso e de cada módulo. Ao trocar duas aulas de posição, a primeira atualização violava essa restrição, pois as duas passavam a ter a mesma ordem momentaneamente. A solução foi mover todas as posições para uma faixa livre (somando 100000) antes de gravar a nova ordem, dentro da mesma transação.

18.4 EXPRESS EM AMBIENTE SERVERLESS

O Express foi feito para rodar como um servidor que escuta uma porta, enquanto a Vercel executa funções sob demanda. O aplicativo passou a ser exportado por um módulo e reaproveitado em dois pontos de entrada: o servidor local e a função serverless. Assim, o código que roda nos testes locais é o mesmo que roda em produção.

18.5 SESSÃO QUE EXPIRA DURANTE O USO

O token de acesso tem duração curta por segurança, o que fazia o usuário receber erros depois de algum tempo com a página aberta. A solução foi centralizar as chamadas no cliente

da API: diante de um 401, ele renova a sessão com o token de renovação e repete a requisição. Várias requisições simultâneas compartilham a mesma renovação, evitando pedidos duplicados.

18.6 MUDANÇA DE REQUISITO COM RASTREABILIDADE

A mudança da regra do certificado mostrou, na prática, por que documentação, código e testes precisam evoluir juntos. Como a regra estava espalhada pela API, pelas telas, pela página inicial, pelos testes e pela documentação, a busca pelo valor antigo em todo o repositório foi parte do processo, e todos os pontos foram atualizados no mesmo Pull Request.

18.7 APRENDIZADOS

O principal aprendizado foi que integrar serviços exige mais do que escrever funções: limites de plataforma, permissões, sessões e consistência entre sistemas diferentes foram responsáveis pelos problemas mais difíceis do projeto. Também ficou claro que os testes automatizados não são um custo extra: foram eles que permitiram mudar regras importantes perto da entrega com segurança.

19 LIMITAÇÕES E TRABALHOS FUTUROS

As limitações atuais são registradas explicitamente, para separar o que está pronto do que é possibilidade:

- o pagamento é simulado: não há integração com um gateway real, e cupons de desconto não foram implementados;
- a avaliação dos cursos pelos alunos (nota e comentário) não foi implementada;
- não há notificação por e-mail quando uma dúvida ou um chamado é respondido;
- há um único papel de administrador, sem instrutores com cursos próprios;
- os indicadores misturam números de demonstração com dados reais;
- não há testes unitários nem testes automatizados da interface, e os testes não rodam automaticamente em cada Pull Request.

As evoluções foram priorizadas pelo impacto e pelos pré-requisitos técnicos:

1. executar a suíte de testes automaticamente em cada Pull Request (integração contínua);
2. integrar um gateway de pagamento (Pix e cartão) e cupons;
3. implementar avaliações dos cursos, substituindo as notas de demonstração;
4. enviar notificações por e-mail de respostas a dúvidas e chamados;
5. permitir vários instrutores, cada um gerenciando os próprios cursos;
6. dividir a API em módulos e documentá-la em OpenAPI;
7. adicionar testes unitários das regras e testes de ponta a ponta da interface.

20 AVALIAÇÃO GLOBAL DO PROJETO

20.1 ATENDIMENTO AO PROBLEMA PROPOSTO

O CursoLab atende ao problema apresentado no Capítulo 2. O acesso ao conteúdo é controlado por matrícula e prazo, o aluno avança em ordem e sabe onde parou, a avaliação é corrigida sem expor o gabarito, o certificado tem uma regra clara e verificável e o suporte está centralizado. O instrutor mantém o conteúdo e atende os alunos pelo backoffice, sem mexer em código.

20.2 COERÊNCIA ENTRE REQUISITOS E ARQUITETURA

A arquitetura é coerente com o porte do projeto e com seus requisitos não funcionais. O monólito modularizado simplifica a implantação, o banco relacional expressa as relações e restrições do domínio, a função serverless elimina a administração de servidor e a decisão de concentrar as regras no servidor atende diretamente aos requisitos de segurança.

20.3 PONTOS FORTES

- sistema publicado e funcionando, com dados persistidos e conteúdo real;
- regras críticas garantidas no servidor e cobertas por 158 verificações automatizadas;
- processo com branches, Pull Requests, migrações versionadas e publicação automática;
- rastreabilidade entre requisitos, código, testes e documentação, demonstrada em mudanças reais;
- limitações registradas de forma explícita.

20.4 PONTOS QUE EXIGEM EVOLUÇÃO

A principal limitação de engenharia é a ausência de integração contínua: os testes existem e são completos, mas dependem de execução manual. Em seguida vêm a concentração da API em um único arquivo e a ausência de testes unitários e de interface. Do ponto de vista funcional, o pagamento real e as avaliações de cursos são as lacunas mais visíveis.

21 CONCLUSÃO

O CursoLab permitiu aplicar na prática os conceitos centrais da disciplina. O projeto partiu de um problema concreto e chegou a uma solução publicada, estruturada por requisitos, arquitetura, modelo de dados, regras de negócio, segurança, testes e documentação.

A análise do projeto à luz dos conteúdos estudados mostra que o desenvolvimento não termina na codificação. A engenharia de requisitos reduziu ambiguidades e permitiu tratar mudanças de forma controlada; a análise e o projeto organizaram a solução em camadas com responsabilidades claras; a verificação, a validação e os testes deram confiança para evoluir; a documentação preservou as decisões; a manutenção aconteceu em paralelo ao desenvolvimento; e o reuso permitiu concentrar o esforço no que é específico do domínio.

Alguns conceitos, apresentados de forma abstrata na disciplina, apareceram de forma concreta no projeto. Requisitos voláteis surgiram na mudança da regra do certificado. Transações surgiram da necessidade de salvar um curso inteiro sem deixá-lo pela metade. A diferença entre verificação e validação ficou clara quando uma regra corretamente implementada se mostrou inadequada. E os testes de regressão mostraram seu valor a cada mudança feita perto da entrega.

O projeto também evidenciou que qualidade de software não vem de uma única tecnologia. Ferramentas modernas ajudam, mas a confiabilidade do sistema veio da combinação entre requisitos claros, regras no servidor, restrições no banco, testes automatizados e documentação atualizada.

Como continuidade, o CursoLab tem espaço para amadurecer tanto funcionalmente, com pagamento real, avaliações e vários instrutores, quanto em engenharia, com integração contínua, modularização da API e testes unitários. Essas evoluções devem preservar os princípios aplicados nesta versão: regras no servidor, mudanças rastreáveis e verificação contínua.

REFERÊNCIAS

SOMMERVILLE, Ian. *Engenharia de Software*. 10. ed. São Paulo: Pearson, 2019.

PRESSMAN, Roger S.; MAXIM, Bruce R. *Engenharia de Software: uma abordagem profissional*. 9. ed. Porto Alegre: AMGH, 2021.

GAMMA, Erich et al. *Padrões de Projeto: soluções reutilizáveis de software orientado a objetos*. Porto Alegre: Bookman, 2000.

SANTOS, Dener. *Princípios de Engenharia de Software*. Material da disciplina. Nova Friburgo: UERJ/IPRJ, 2026.

SUPABASE. *Supabase Documentation*. Disponível em: https://supabase.com/docs. Acesso em: 7 out. 2026.

VERCEL. *Vercel Documentation*. Disponível em: https://vercel.com/docs. Acesso em: 7 out. 2026.

REACT. *React Documentation*. Disponível em: https://react.dev. Acesso em: 7 out. 2026.

EXPRESS. *Express Documentation*. Disponível em: https://expressjs.com. Acesso em: 7 out. 2026.

CURSOLAB. *Repositório do projeto: código-fonte, migrações, testes e documentação*. GitHub, 2026.